7CS033 Assignment Task 1

Complete the following tasks.

You should comment on your code where appropriate. Failing to add comments may result in a mark deduction.

You should write any explainations in the report cell if the task explicitly asks for this or if you wish to expand upon your solution.

Ensure you have ran your code cell and printed any of the required outputs. If necessary you can use more than one code cell.

TASK 1.1: Write a code to receive an optional text, capitalise all words in the text and print them (2%)

In [3]:
```
############# WRITE THE CODE IN THIS CELL ####################
#Solution

# Ask the user to enter text (optional input)
# If the user just presses Enter, use a default sentence
text1 = input("Enter a sentence: ")

# Create an empty list to store capitalized words
output=[];

if text1 == "":
    text1 = "welcome to assessment one"

# Split the sentence into individual words
for word in text1.split():

    # Capitalize the first letter of each word
    capitalized_word= word.capitalize()

    # Add the capitalized word to the list
    output.append(capitalized_word)
    #print(word);

print(output)

# Join the words back into a full sentence
sentence = ' '.join(output);

# Print the final result
print("Capitalized Sentence:", sentence)
```

```
['Welcome', 'To', 'Assessment', 'One']
Capitalized Sentence: Welcome To Assessment One
```

############ WRITE EXPLANATIONS HERE (IF APPLICABLE)
##############

TASK 1.2: Write a program that prompts the user to input the radius of a circle and calculates its area (2%)

In [6]:
```
############ WRITE THE CODE IN THIS CELL ###################

# Import the math module to use the value of pi
import math


# Prompt the user to enter the radius (allow decimal values)
inputRadius = float(input("Enter Radius of the circle: "))

# Calculate the area using the formula: πr²
circleArea = math.pi * (inputRadius ** 2)


# Display the result
print("The area of the circle is:", circleArea)
```

```
The area of the circle is: 503912.24703396624
```

In [ ]:
```

```

############ WRITE EXPLANATIONS HERE (IF APPLICABLE)
##############

TASK 1.3: Download and import the 'adult' dataset from Canvas. Write a function to split the dataset column-wise into two halves and swap the first half with the second half (3%)

In [9]:
```
############ WRITE THE CODE IN THIS CELL ###################

import pandas as pd
import numpy as np

# Read the CSV file into a DataFrame
df = pd.read_csv('adult.csv');


# Define a function to split and swap columns
def swap_column_halves(df):

    # Get total number of columns
    num_cols = df.shape[1]

    # Find the midpoint of the columns
```

```python
    mid = num_cols // 2

    # Split the columns into two halves
    first_half = df.iloc[:, :mid]
    second_half = df.iloc[:, mid:]

    # Swap the halves and concatenate them
    swapped_df = pd.concat([second_half, first_half], axis=1)

    return swapped_df



# Call the function
swapped_df = swap_column_halves(df)

# Display the swapped dataframe
swapped_df
```

Out[9]:

	relationship	race	sex	capital-gain	capital-loss	hours-per-week	native-country	income
0	Not-in-family	White	Male	2174	0	40	United-States	<=50K
1	Husband	White	Male	0	0	13	United-States	<=50K
2	Not-in-family	White	Male	0	0	40	United-States	<=50K
3	Husband	Black	Male	0	0	40	United-States	<=50K
4	Wife	Black	Female	0	0	40	Cuba	<=50K
...	...	...	...	...	...	...	...	...
48837	Not-in-family	White	Female	0	0	36	United-States	<=50K.
48838	Other-relative	Black	Male	0	0	40	United-States	<=50K.
48839	Husband	White	Male	0	0	50	United-States	<=50K.
48840	Own-child	Asian-Pac-Islander	Male	5455	0	40	United-States	<=50K.
48841	Husband	White	Male	0	0	60	United-States	>50K.

48842 rows × 15 columns

########### WRITE EXPLANATIONS HERE (IF APPLICABLE) ############

Task 1.4: Write a function that takes the names of two numerical columns as input and compares their values for all rows. If the value in the first column is greater than the value in the second column, the function should return True; otherwise, it should return False. The function should append a new column to the dataset to store the comparison results for all rows. Apply the function to the 'age' and 'hours-per-week' columns in the adult dataset and print the resulting dataset (3%)

In [12]:
```
############# WRITE THE CODE IN THIS CELL ####################

df = pd.read_csv('adult.csv');



# Define a function that:
# 1. Takes a dataframe
# 2. Takes two numerical column names as input
# 3. Compares their values row-by-row
# 4. Appends a new column with True/False results
def isGreaterThan(dataframe, col1, col2):

    # Perform vectorized comparison:
    # Returns True if value in col1 > value in col2, otherwise False
    dataframe['isGreaterThan'] = dataframe[col1] > dataframe[col2]

    # Return the updated dataframe
    return dataframe

# Apply the function to compare 'age' and 'hours-per-week'
df = isGreaterThan(df, 'age', 'hours-per-week')

# Display the resulting dataset
df
```

Out[12]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	rel
0	39	State-gov	77516	Bachelors	13	Never-married	Adm-clerical	Nc
1	50	Self-emp-not-inc	83311	Bachelors	13	Married-civ-spouse	Exec-managerial	
2	38	Private	215646	HS-grad	9	Divorced	Handlers-cleaners	Nc
3	53	Private	234721	11th	7	Married-civ-spouse	Handlers-cleaners	
4	28	Private	338409	Bachelors	13	Married-civ-spouse	Prof-specialty	
...	...	...	...	...	...	...	...	
48837	39	Private	215419	Bachelors	13	Divorced	Prof-specialty	Nc
48838	64	NaN	321403	HS-grad	9	Widowed	NaN	
48839	38	Private	374983	Bachelors	13	Married-civ-spouse	Prof-specialty	
48840	44	Private	83891	Bachelors	13	Divorced	Adm-clerical	
48841	35	Self-emp-inc	182148	Bachelors	13	Married-civ-spouse	Exec-managerial	

48842 rows × 16 columns

############ WRITE EXPLANATIONS HERE (IF APPLICABLE) #############

Task 1.1: Demonstrates string manipulation by capitalizing words in a sentence.
Task 1.2: Applies mathematical calculations to compute the area of a circle using user input.
Task 1.3: Performs dataframe column operations by splitting and swapping dataset columns.
Task 1.4: Uses functions and pandas operations to compare numerical columns and append logical (True/False) results to the dataset

In [ ]:

7CS033 Assignment Task 2

Complete the following tasks.

You should comment on your code where appropriate. Failing to add comments may result in a mark deduction.

You should write any explainations in the report cell if the task explicitly asks for this or if you wish to expand upon your solution.

Ensure you have ran your code cell and printed any of the required outputs. If necessary you can use more than one code cell.

TASK 2.1: Download the adult dataset from Canvas, import it into Jupyter Notebook, and perform a complete data preprocessing. Your solution should include at least the following steps (completing the report cell is required):

a. Change the display setting and print the first 100 rows.

b. Print the dataset infromation.

c. Write a Python script to display the number of Null values in each column. Handle the Null values using an appropriate method, and explain the rationale behind your choice in the report cell. Print the number of null values again at the end to ensure that no null values remain in the processed dataset.

d. Assume we want to predict the income column. Encode the data using the appropriate method.

e. Normalise the data.

NOTE: You must add comments to your code. Failure to do so will result in a mark deduction.

In [84]:
```
############# WRITE YOUR CODE IN THIS CELL (IF APPLICABLE)####################

import pandas as pd
import numpy as np
```

```python
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
```

In [ ]:

```python

```

In [88]:

```python
#Change display setting and print first 100 rows

# Set pandas to display all columns
pd.set_option('display.max_columns', None)

# Set pandas to display up to 100 rows
pd.set_option('display.max_rows', 100)

# Load the dataset (ensure adult.csv is in your directory)
adult_df = pd.read_csv('adult_T3.csv')

# Print first 100 rows
print("First 100 rows of the dataset:")
print(adult_df.head(10))
```

```
First 100 rows of the dataset:
   age  workclass  fnlwgt     education  education-num      marital-status  \
0   29    Private  216481       Masters             14  Married-civ-spouse
1   36    Private  280570  Some-college             10  Married-civ-spouse
2   25          ?  100903     Bachelors             13  Married-civ-spouse
3   47    Private  145636     Assoc-voc             11  Married-civ-spouse
4   33    Private  119422       HS-grad              9  Married-civ-spouse
5   37    Private  186808       HS-grad              9       Never-married
6   34    Private  339142       HS-grad              9           Separated
7   38    Private  101387       HS-grad              9  Married-civ-spouse
8   62    Private  166691       HS-grad              9            Divorced
9   50  Local-gov   50178  Some-college             10  Married-civ-spouse

          occupation relationship   race     sex  capital-gain  capital-loss  \
0    Exec-managerial         Wife  White  Female             0             0
1       Craft-repair      Husband  White    Male             0             0
2                  ?         Wife  White  Female             0             0
3  Handlers-cleaners      Husband  White    Male             0             0
4    Exec-managerial      Husband  White    Male             0             0
5              Sales    Unmarried  White    Male             0             0
6  Handlers-cleaners    Unmarried  White  Female             0             0
7      Other-service         Wife  White  Female             0             0
8    Exec-managerial    Unmarried  White  Female             0             0
9       Craft-repair      Husband  White    Male          4064             0

   hours-per-week native-country income
0              40  United-States   >50K
1              45  United-States  <=50K
2              25  United-States  <=50K
3              48  United-States   >50K
4              40  United-States  <=50K
5              40  United-States  <=50K
6              40  United-States  <=50K
7              43  United-States  <=50K
8              40  United-States  <=50K
9              55  United-States  <=50K
```

In [ ]:

In [92]:

```
#Print dataset information
print("Dataset Information:")
print(adult_df.info())
```

```
Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 15 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   age             10000 non-null  int64
 1   workclass       9418 non-null   str
 2   fnlwgt          10000 non-null  int64
 3   education       10000 non-null  str
 4   education-num   10000 non-null  int64
 5   marital-status  10000 non-null  str
 6   occupation      9417 non-null   str
 7   relationship    10000 non-null  str
 8   race            10000 non-null  str
 9   sex             10000 non-null  str
 10  capital-gain    10000 non-null  int64
 11  capital-loss    10000 non-null  int64
 12  hours-per-week  10000 non-null  int64
 13  native-country  9803 non-null   str
 14  income          10000 non-null  str
dtypes: int64(6), str(9)
memory usage: 1.9 MB
None
```

Handle the Null values

In [91]:

```python
# Replace '?' with NaN so pandas recognises them as missing
adult_df.replace('?', np.nan, inplace=True)

# Display null values BEFORE cleaning
print("Null values per column BEFORE cleaning:")
print(adult_df.isna().sum())
```

```
Null values per column BEFORE cleaning:
age                 0
workclass         582
fnlwgt              0
education           0
education-num       0
marital-status      0
occupation        583
relationship        0
race                0
sex                 0
capital-gain        0
capital-loss        0
hours-per-week      0
native-country    197
income              0
dtype: int64
```

Fill Missing Values

In [93]:
```
#Select numerical columns
num_cols = adult_df.select_dtypes(include=['int64', 'float64']).columns

#Fill missing numerical values with mean
for col in num_cols:
    adult_df[col] = adult_df[col].fillna(adult_df[col].mean())
```

In [ ]:
```

```

In [95]:
```
# Select Text  columns
cat_cols = adult_df.select_dtypes(include=['object']).columns

# Fill missing Text values with mode (most frequent value)
for col in cat_cols:
    adult_df[col] = adult_df[col].fillna(adult_df[col].mode()[0])
```

```
/var/folders/6b/x27jrjp92kn9r5_84c_ywfhcsgm8y7/T/ipykernel_33394/1518027836.p
y:2: Pandas4Warning: For backward compatibility, 'str' dtypes are included by s
elect_dtypes when 'object' dtype is specified. This behavior is deprecated and
will be removed in a future version. Explicitly pass 'str' to `include` to sele
ct them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-m
igration-select-dtypes for details on how to write code that works with pandas
2 and 3.
  cat_cols = adult_df.select_dtypes(include=['object']).columns
```

In [ ]:
```
#Numerical missing values were replaced with the mean, and text-based missing
#replaced with the mode to preserve data distribution and avoid losing records
```

In [96]:
```
# Verify that no missing values remain
print("Null values per column AFTER cleaning:")
print(adult_df.isna().sum())
```

```
Null values per column AFTER cleaning:
age               0
workclass         0
fnlwgt            0
education         0
education-num     0
marital-status    0
occupation        0
relationship      0
race              0
sex               0
capital-gain      0
capital-loss      0
hours-per-week    0
native-country    0
income            0
dtype: int64
```

Encode the Data

In [97]:
```
#Encode the data
# Create LabelEncoder object
le = LabelEncoder()

# Encode target variable 'income'
adult_df['income'] = le.fit_transform(adult_df['income'])
```

In [ ]:
```

```

In [98]:
```
# Encode remaining Text columns
for col in cat_cols:
    if col != 'income':   # income already encoded
        adult_df[col] = le.fit_transform(adult_df[col])

print("Dataset after encoding:")
print(adult_df.head())
```

```
Dataset after encoding:
   age  workclass  fnlwgt  education  education-num  marital-status  \
0   29          3  216481         12             14               2
1   36          3  280570         15             10               2
2   25          3  100903          9             13               2
3   47          3  145636          8             11               2
4   33          3  119422         11              9               2

   occupation  relationship  race  sex  capital-gain  capital-loss  \
0           3             5     4    0             0             0
1           2             0     4    1             0             0
2           9             5     4    0             0             0
3           5             0     4    1             0             0
4           3             0     4    1             0             0

   hours-per-week  native-country  income
0              40              37       1
1              45              37       0
2              25              37       0
3              48              37       1
4              40              37       0
```

Normalise the Data

In [99]:
```
# Create scaler object
scaler = MinMaxScaler(feature_range=(0, 1))

# Apply scaling
scaled_data = scaler.fit_transform(adult_df)
```

In [100…
```
# Convert back to DataFrame
adult_scaled_df = pd.DataFrame(scaled_data, columns=adult_df.columns)
```

```
print("First 5 rows after normalisation:")
print(adult_scaled_df.head())
```

```
First 5 rows after normalisation:
        age  workclass    fnlwgt  education  education-num  marital-status  \
0  0.164384   0.428571  0.150828   0.800000       0.866667        0.333333
1  0.260274   0.428571  0.198167   1.000000       0.600000        0.333333
2  0.109589   0.428571  0.065457   0.600000       0.800000        0.333333
3  0.410959   0.428571  0.098499   0.533333       0.666667        0.333333
4  0.219178   0.428571  0.079136   0.733333       0.533333        0.333333

   occupation  relationship  race  sex  capital-gain  capital-loss  \
0    0.230769           1.0   1.0  0.0           0.0           0.0
1    0.153846           0.0   1.0  1.0           0.0           0.0
2    0.692308           1.0   1.0  0.0           0.0           0.0
3    0.384615           0.0   1.0  1.0           0.0           0.0
4    0.230769           0.0   1.0  1.0           0.0           0.0

   hours-per-week  native-country  income
0        0.397959        0.948718     1.0
1        0.448980        0.948718     0.0
2        0.244898        0.948718     0.0
3        0.479592        0.948718     1.0
4        0.397959        0.948718     0.0
```

In [ ]:

############# WRITE YOUR REPORT IN THIS CELL (IF APPLICABLE)################### The dataset was explored and cleaned by replacing missing values and filling them using the mean (for numerical columns) and mode (for text columns). Numerical missing values were replaced with the mean, and text-based missing values were replaced with the mode to preserve data distribution and avoid losing records Finally, the data was normalised using MinMaxScaler and is ready for modelling.

7CS033 Assignment Task 3

Complete the following tasks.

You should comment on your code where appropriate. Failing to add comments may result in a mark deduction.

You should write any explainations in the report cell if the task explicitly asks for this or if you wish to expand upon your solution.

Ensure you have ran your code cell and printed any of the required outputs. If necessary you can use more than one code cell.

Task

TASK 3.1: Apply four classifiers discussed in Lecture 3, i.e. Support Vector Machine (SVM), Decision Tree (DT), Random Forest (RF), and K-nearest neighbours (KNN) classifiers to the adult_T3 dataset available on Canvas to predict the income column. Calculate the confusion matrix and evaluation metrics for all classifiers (25%):

NOTE1: To decrease the processing time, use an ordinal encoder for both nominal and ordinal input columns. Use label encoding for the target feature.

NOTE2 You are expected to improve your models to get as high accuracy as possible.

NOTE3: You should add comments on your code wherever necessary and briefly explain what the code is doing

NOTE4: Completing the report cell is NOT required. However, you can add explanations to the report cell if you need to do so.

NOTE6: You will still get some marks if your code doesn't run, but you have written some codes and have added comments on the code.

In [82]:
```
############# WRITE YOUR CODE IN THIS CELL (IF APPLICABLE)####################

# Importing necessary Libraries which we would use
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OrdinalEncoder

from sklearn.model_selection import train_test_split
```

```python
from sklearn import svm
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn import metrics
```

In [86]:

```python
#Solution

df = pd.read_csv('adult_T3.csv')
df2 = df

#print a sample of the 5 data
print(df.sample(5))

#identify the number of missing values i
print(pd.isnull(df).sum());
```

```
      age         workclass  fnlwgt     education  education-num  \
1565   22           Private  362309  Some-college             10
1961   19           Private  261259       HS-grad              9
5894   38           Private  203763       HS-grad              9
6517   49  Self-emp-not-inc  144351  Some-college             10
4472   28           Private   60374       HS-grad              9

          marital-status         occupation   relationship   race     sex  \
1565       Never-married      Other-service      Own-child  White    Male
1961       Never-married  Handlers-cleaners      Own-child  White    Male
5894            Divorced       Adm-clerical  Not-in-family  White  Female
6517  Married-civ-spouse              Sales        Husband  White    Male
4472  Married-civ-spouse      Other-service      Own-child  White  Female

      capital-gain  capital-loss  hours-per-week native-country income
1565             0             0              35  United-States  <=50K
1961             0             0              20  United-States  <=50K
5894             0             0              40  United-States  <=50K
6517             0             0              25  United-States  <=50K
4472             0             0              35  United-States  <=50K
age                 0
workclass         175
fnlwgt              0
education           0
education-num       0
marital-status      0
occupation        175
relationship        0
race                0
sex                 0
capital-gain        0
capital-loss        0
hours-per-week      0
native-country     61
income              0
dtype: int64
```

In [87]:

```
#Replace ? with empty in dataset
df2.replace("?", np.nan, inplace=True)
```

In [88]:

```
# Fill missing occupation values using education as a reference (related colum
df2["occupation"] = df2.groupby("education")["occupation"].transform(

    # x is the 'occupation' column for one education group
    lambda x: x.fillna(

        # If this education group has a most common value (mode),
        # use that to fill the missing values
        x.mode().iloc[0]

        # But only if the mode is not empty
        if not x.mode().empty
```

```
        # If this group has no valid occupation at all,
        # fall back to the global most common occupation
        else df2["occupation"].mode().iloc[0]
    )
)

print(pd.isnull(df2).sum());
#print(df.sample(5))
```

```
age                 0
workclass         582
fnlwgt              0
education           0
education-num       0
marital-status      0
occupation          0
relationship        0
race                0
sex                 0
capital-gain        0
capital-loss        0
hours-per-week      0
native-country    197
income              0
dtype: int64
```

In [89]:

```
# Fill missing native-country values using race as a reference (related column
df2["native-country"] = df2.groupby("race")["native-country"].transform(

    # x is the 'native-country' column for one race group
    lambda x: x.fillna(

        # If this race group has a most common value (mode),
        # use that to fill the missing values
        x.mode().iloc[0]

        # But only if the mode is not empty
        if not x.mode().empty

        # If this group has no valid native-country at all,
        # fall back to the global most common native-country
        else df2["native-country"].mode()[0])
);

print(pd.isnull(df2).sum());
```

```
age                 0
workclass         582
fnlwgt              0
education           0
education-num       0
marital-status      0
occupation          0
relationship        0
race                0
sex                 0
capital-gain        0
capital-loss        0
hours-per-week      0
native-country      0
income              0
dtype: int64
```

In [90]:

```
# Fill missing workclass values using race as a reference (related column)
df2["workclass"] = df2.groupby("occupation")["workclass"].transform(

    # x is the 'workclass' column for one race group
    lambda x: x.fillna(

        # If this occupation group has a most common value (mode),
        # use that to fill the missing values
        x.mode().iloc[0]

        # But only if the mode is not empty
        if not x.mode().empty

        # If this group has no valid workclass at all,
        # fall back to the global most common native-country
        else df2["workclass"].mode()[0])
);

print(pd.isnull(df2).sum());
```

```
age               0
workclass         0
fnlwgt            0
education         0
education-num     0
marital-status    0
occupation        0
relationship      0
race              0
sex               0
capital-gain      0
capital-loss      0
hours-per-week    0
native-country    0
income            0
dtype: int64
```

In [92]:
```
#label encoding for income, Sex, Workclass and other colums with string values

'''
df2['income'] = le.fit_transform(df2['income'])

df2['sex'] = le.fit_transform(df2['sex'])

df2['workclass'] = le.fit_transform(df2['workclass'])

df2['education'] = le.fit_transform(df2['education'])

df2['marital-status'] = le.fit_transform(df2['marital-status'])

df2['relationship'] = le.fit_transform(df2['relationship'])


df2['race'] = le.fit_transform(df2['race'])
df2['native-country'] = le.fit_transform(df2['native-country'])
df2['relationship'] = le.fit_transform(df2['relationship'])

df2['occupation'] = le.fit_transform(df2['occupation'])
'''




encoder = OrdinalEncoder()
categorical_cols = df2.select_dtypes(include='object').columns
df2[categorical_cols] = encoder.fit_transform(df2[categorical_cols])

le = LabelEncoder()
df2['income'] = le.fit_transform(df2['income'])

df2.head()
```

Out[92]:

	age	workclass	fnlwgt	education	education-num	marital-status	occupation	relation
0	29	3.0	216481	12.0	14	2.0	3.0	
1	36	3.0	280570	15.0	10	2.0	2.0	
2	25	3.0	100903	9.0	13	2.0	9.0	
3	47	3.0	145636	8.0	11	2.0	5.0	
4	33	3.0	119422	11.0	9	2.0	3.0	

In [ ]:

In [93]:
```
#delete  education-num and fnlwgt
df2 = df2.drop(['education-num', 'fnlwgt'], axis = 1);
df2.head()
```

Out[93]:

	age	workclass	education	marital-status	occupation	relationship	race	sex	cap
0	29	3.0	12.0	2.0	3.0	5.0	4.0	0.0	
1	36	3.0	15.0	2.0	2.0	0.0	4.0	1.0	
2	25	3.0	9.0	2.0	9.0	5.0	4.0	0.0	
3	47	3.0	8.0	2.0	5.0	0.0	4.0	1.0	
4	33	3.0	11.0	2.0	3.0	0.0	4.0	1.0	

In [95]:

```
#get value for x and y
X = df2.drop('income', axis=1)
y = df2['income']

#Scale the entire dataset without income
scaler = MinMaxScaler()
X = scaler.fit_transform(X)


X
```

Out[95]:

```
array([[0.16438356, 0.42857143, 0.8       , ..., 0.        , 0.39795918,
        0.94871795],
       [0.26027397, 0.42857143, 1.        , ..., 0.        , 0.44897959,
        0.94871795],
       [0.10958904, 0.42857143, 0.6       , ..., 0.        , 0.24489796,
        0.94871795],
       ...,
       [0.17808219, 0.42857143, 0.73333333, ..., 0.        , 0.39795918,
        0.94871795],
       [0.32876712, 0.71428571, 0.53333333, ..., 0.        , 0.39795918,
        0.94871795],
       [0.05479452, 0.42857143, 1.        , ..., 0.        , 0.39795918,
        0.94871795]])
```

In [51]:

```
#Get X and Y dataset from df2
```

In [96]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, ran
```

In [99]:

```
# C controls regularization strength (Helps to to fit training data better)
# Using RBF kernel for non-linear decision boundaries
# random_state ensures reproducibility
SVM = svm.SVC(kernel='rbf', C=2, random_state=42)


# Decision Tree classifier
# random_state ensures reproducible results
DT = DecisionTreeClassifier(random_state=42)
```

```
# n_estimators = number of trees in the forest
# max_depth limits tree depth to prevent overfitting
# random_state ensures reproducibility
RF = RandomForestClassifier(n_estimators=200, max_depth=10, random_state=42)


# n_neighbors defines how many nearby points influence classification
# Larger values reduce noise but may smooth decision boundaries
KNN = KNeighborsClassifier(n_neighbors=7)


SVM.fit(X_train, y_train)
DT.fit(X_train, y_train)
RF.fit(X_train, y_train)
KNN.fit(X_train, y_train)
```

Out[99]:

```
▾ KNeighborsClassifier
KNeighborsClassifier(n_neighbors=7)
```

In [100…

```
y
```

Out[100…

```
0       1
1       0
2       0
3       1
4       0
       ..
9995    0
9996    0
9997    0
9998    1
9999    0
Name: income, Length: 10000, dtype: int64
```

In [ ]:

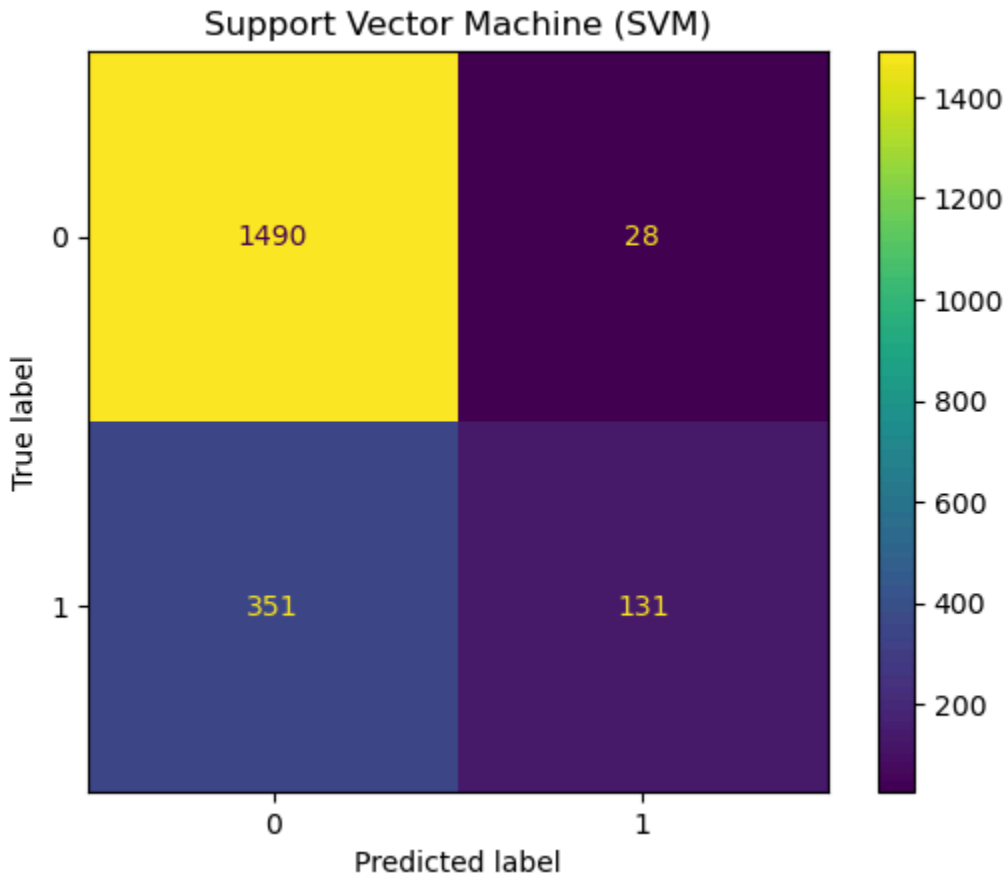
```
#first make predictions using our trained models on the test data
#Now we can generate the confusion matrices
```

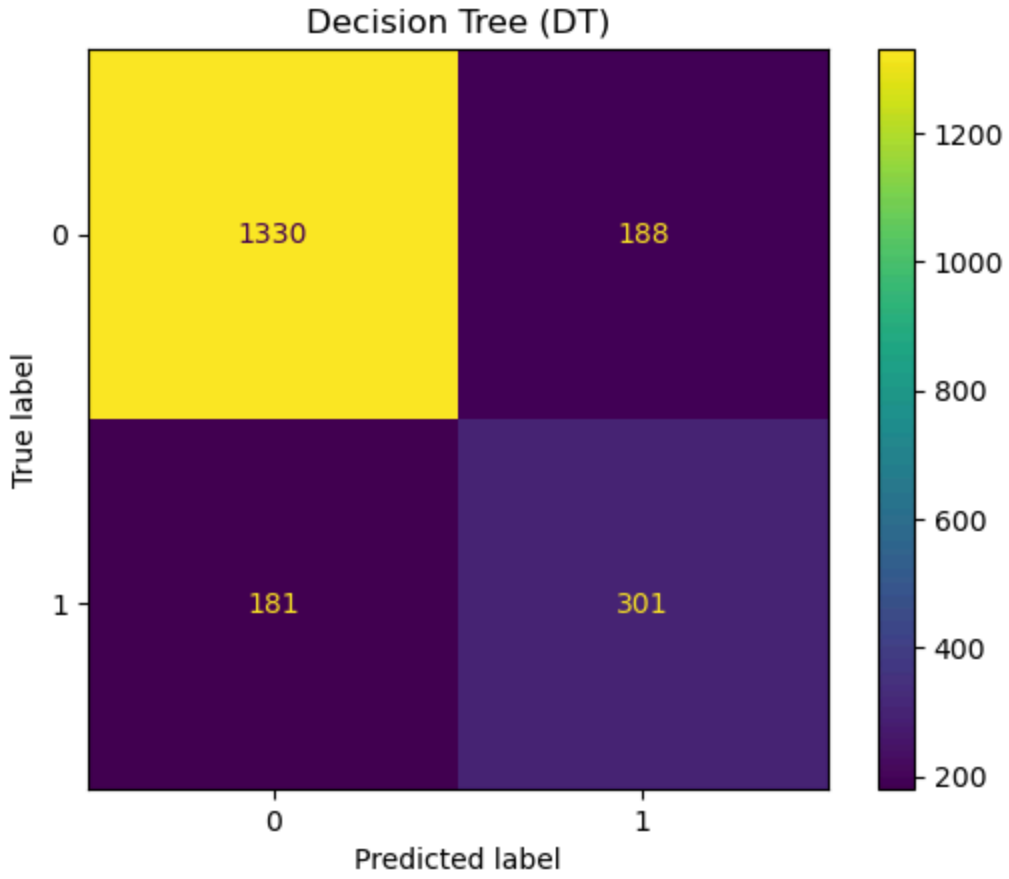
In [59]:

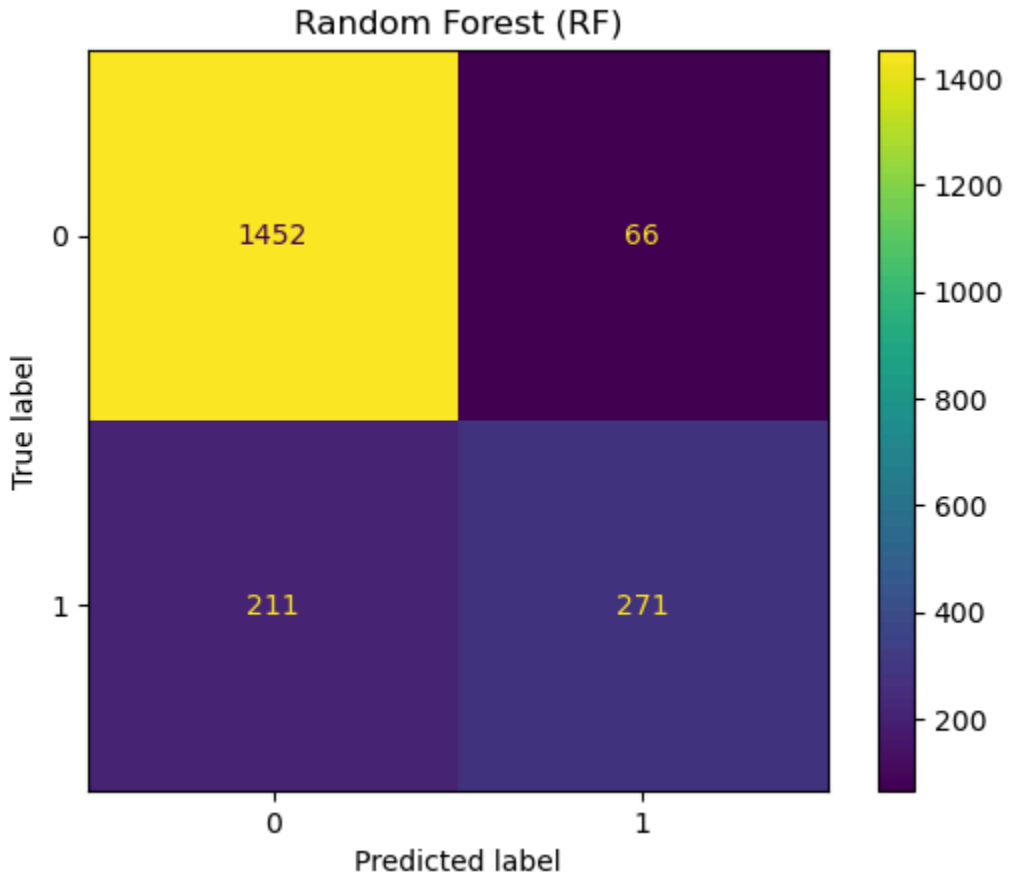
```
#A reusable helper function to Plot confusion matrix for all classifiers
def plot_confusion_for_all_models(model, X_test, y_test, title):

    #first make predictions using our trained models on the test data
    preds = model.predict(X_test)
    cm = confusion_matrix(y_test, preds, labels=model.classes_)

    #Now we can generate the confusion matrices
    disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                                  display_labels=model.classes_)
    disp.plot()
    plt.title(title)
    plt.show()
```

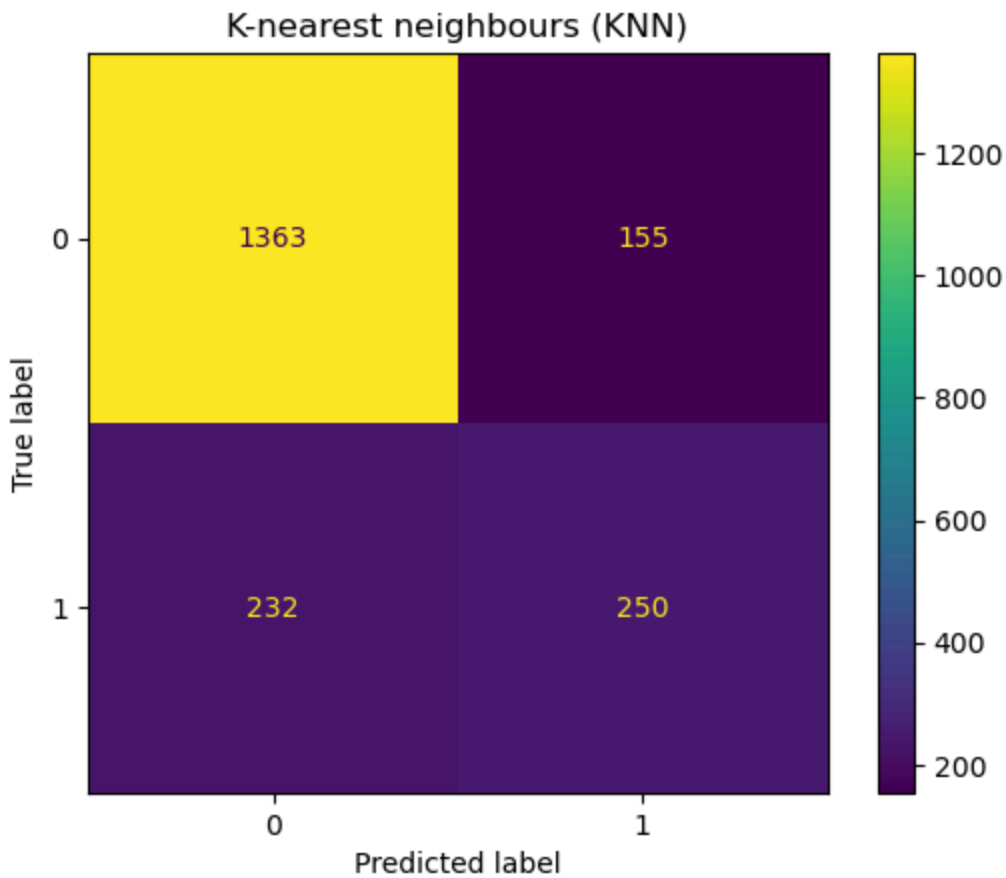
In [101…

```
#Plot for
#Support Vector Machine (SVM), Decision Tree (DT), Random Forest (RF), and K-n
plot_confusion_for_all_models(SVM,X_test,y_test,'Support Vector Machine (SVM)'
plot_confusion_for_all_models(DT,X_test,y_test,'Decision Tree (DT)');
plot_confusion_for_all_models(RF,X_test,y_test,'Random Forest (RF)');
plot_confusion_for_all_models(KNN,X_test,y_test,'K-nearest neighbours (KNN)');
```




Decision Tree (DT)
1330
188
181
301
True label
Predicted label
0
1
0
1
1200
1000
800
600
400
200


Random Forest (RF)
1452
66
211
271
True label
Predicted label
0
1
0
1
1400
1200
1000
800
600
400
200



In [ ]:

In [102…

```
#A function to display accuracy, precision, recall and F1 score for the models
def confusion_Metrix_Analysis(y_test,X_test,model):
    model_prediction = model.predict(X_test)
    print(model,'METRICS')
    print('Accuracy: %.2f' %metrics.accuracy_score(y_test, model_prediction))
    print('Precision: %.2f' %metrics.precision_score(y_test, model_prediction)
    print('Recall: %.2f' %metrics.recall_score(y_test, model_prediction))
    print('F1 Score: %.2f \n' %metrics.f1_score(y_test, model_prediction))
```

In [ ]:

In [103…

```
from sklearn.metrics import classification_report
model_prediction = SVM.predict(X_test)
print(classification_report(y_test, model_prediction))
```

```
              precision    recall  f1-score   support

           0       0.81      0.98      0.89      1518
           1       0.82      0.27      0.41       482

    accuracy                           0.81      2000
   macro avg       0.82      0.63      0.65      2000
weighted avg       0.81      0.81      0.77      2000
```

In [ ]:

In [104…

```
confusion_Metrix_Analysis(y_test,X_test,SVM);
confusion_Metrix_Analysis(y_test,X_test,DT);
confusion_Metrix_Analysis(y_test,X_test,RF);
confusion_Metrix_Analysis(y_test,X_test,KNN);
```

```
SVC(C=2, random_state=42) METRICS
Accuracy: 0.81
Precision: 0.82
Recall: 0.27
F1 Score: 0.41

DecisionTreeClassifier(random_state=42) METRICS
Accuracy: 0.82
Precision: 0.62
Recall: 0.62
F1 Score: 0.62

RandomForestClassifier(max_depth=10, n_estimators=200, random_state=42) METRICS
Accuracy: 0.86
Precision: 0.80
Recall: 0.56
F1 Score: 0.66

KNeighborsClassifier(n_neighbors=7) METRICS
Accuracy: 0.81
Precision: 0.62
Recall: 0.52
F1 Score: 0.56
```

############## WRITE YOUR REPORT IN THIS CELL (IF APPLICABLE)#################### The Random Forest classifier is the best model among the four tested. It achieved the highest accuracy (0.86) and the highest F1 score (0.66), which shows it provides the best balance between precision and recall. While SVM had high precision, its recall was very low, meaning it missed many true positive cases. Decision Tree and KNN performed moderately but did not match the overall balance of Random Forest. The strong performance of Random Forest is due to its ensemble approach, where multiple decision trees work together to reduce overfitting and improve prediction stability. Therefore, Random Forest is the most reliable model for predicting income in this dataset.

In [ ]:

7CS033 Assignment Task 4

Complete the following tasks.

You should comment on your code where appropriate. Failing to add comments may result in a mark deduction.

You should write any explainations in the report cell if the task explicitly asks for this or if you wish to expand upon your solution.

Ensure you have ran your code cell and printed any of the required outputs. If necessary you can use more than one code cell.

Tasks

TASK 4.1: Download the adult_T4 dataset. Apply K-Means and Hierarchical clustering to three optional columns in the dataset. Find the optimum number of clusters for both clustering methods (10%).

NOTE: You should comment on your code wherever necessary and briefly explain what the code is doing

In [103…

```
############# WRITE YOUR CODE IN THIS CELL (IF APPLICABLE)####################

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.decomposition import PCA


from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OrdinalEncoder

from mpl_toolkits.mplot3d import Axes3D
```

In [121…

```
df = pd.read_csv('adult_T4.csv')

d1 = df['age'] #target variable 1
d2 = df['education-num'] #target variable 2
d3 = df['hours-per-week'] #target variable 3
```

In [122…

```
# Select three numerical columns for clustering
# These variables are continuous and suitable for distance-based clustering

X = pd.concat([d1,d2,d3], axis = 1); #get our 3 input variables
#X = X.dropna()
```

In [123…

```
# Remove missing values (clustering cannot handle NaNs)
X = X.dropna()
```

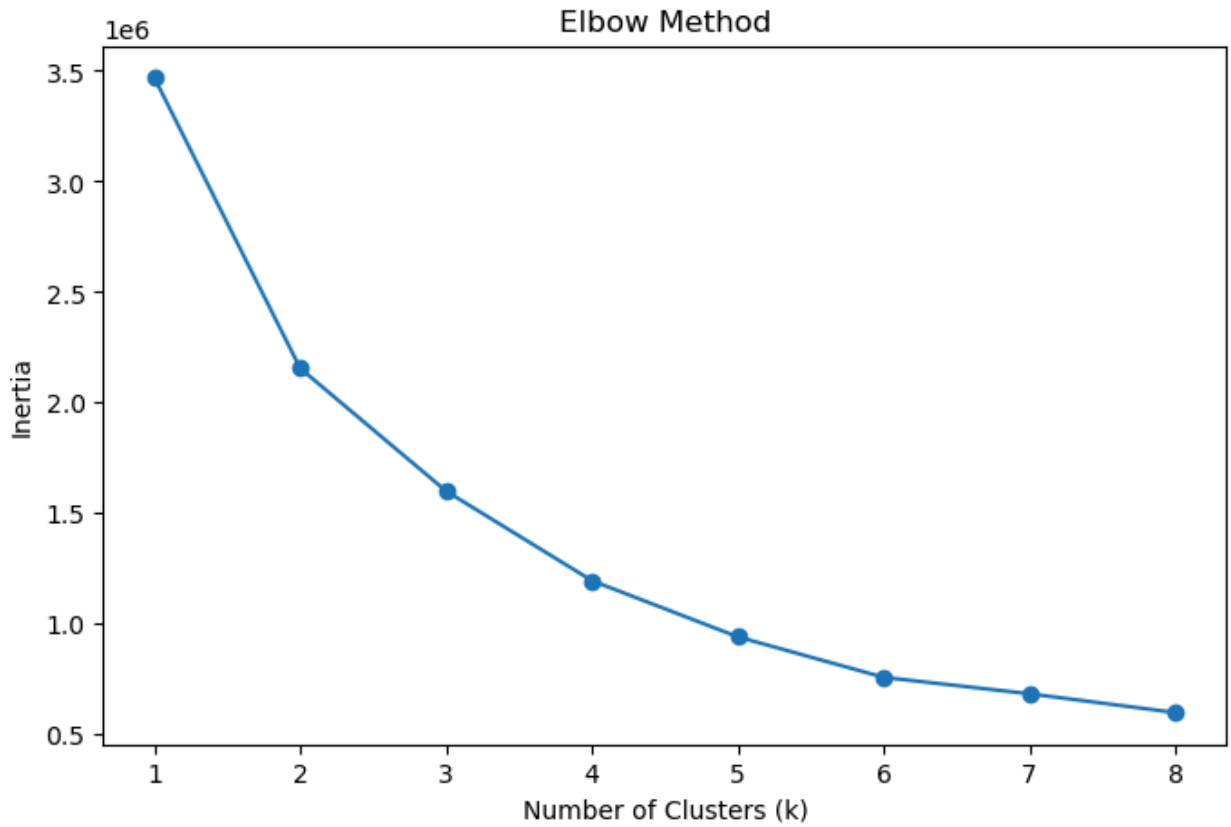
In [124…

```
# Scaling is important because clustering uses distance metrics
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

K-MEANS CLUSTERING

In [125…

```
# --- Elbow Method ---
# Used to identify the point where adding more clusters
# does not significantly reduce inertia

inertia = []


for k in range(1, 9):
    km = KMeans(n_clusters = k, random_state = 42)
    km.fit(X)
    inertia.append(km.inertia_)

#plot the elbow curve
plt.figure(figsize=(8,5))
plt.plot(range(1,9), inertia, marker = 'o')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method')
plt.show()
```

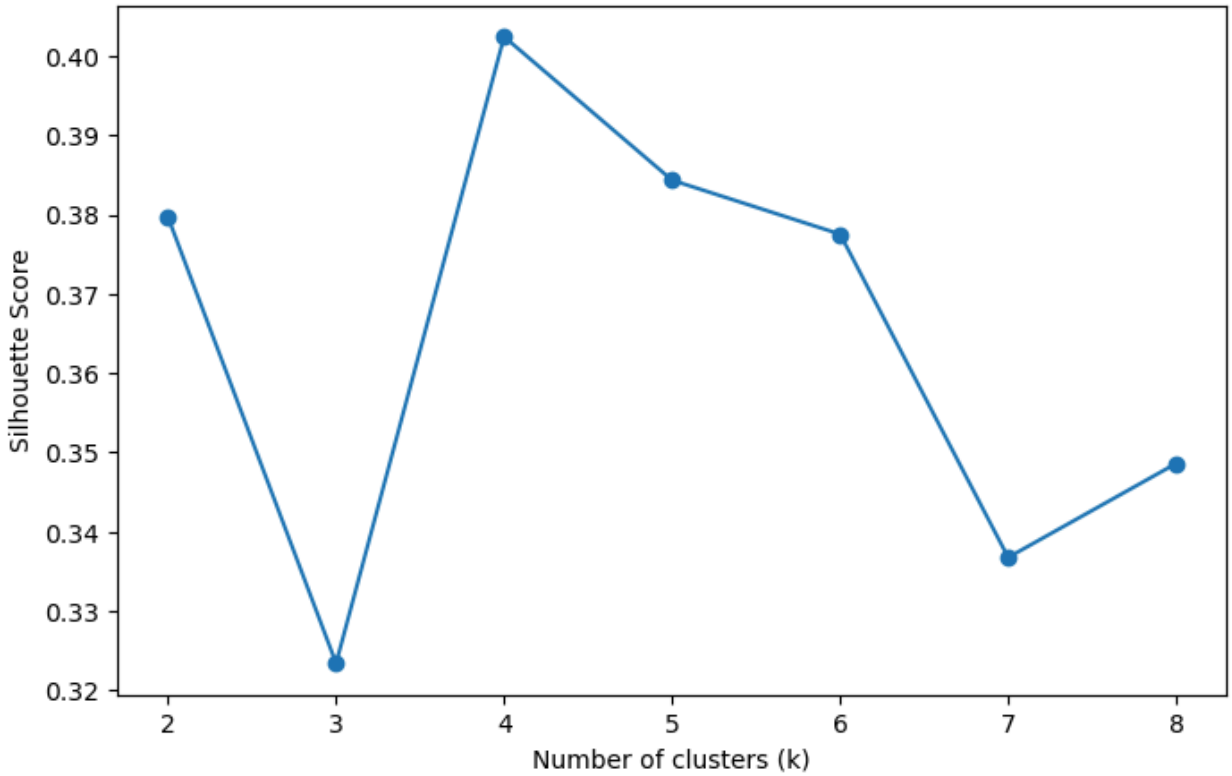


In [ ]:

In [126…

```
# Silhouette Method
# Measures how well points fit within their clusters

sil_scores = []

#
for k in range (2, 9):
    km = KMeans(n_clusters = k, random_state = 42)
    labels = km.fit_predict(X)
    score = silhouette_score(X, labels)
    sil_scores.append(silhouette_score(X, labels))
    print(f"k={k}, Silhouette Score={score}")


plt.figure(figsize = (8, 5))
plt.plot(range(2,9), sil_scores, marker = 'o')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Silhouette Score')
plt.show()
```

```
k=2, Silhouette Score=0.37965490443771743
k=3, Silhouette Score=0.323411684363288
k=4, Silhouette Score=0.4024432969339395
k=5, Silhouette Score=0.3843419106780839
k=6, Silhouette Score=0.3775365397877103
k=7, Silhouette Score=0.3367647194464343
k=8, Silhouette Score=0.3486205456416944
```



In [129…

```
# Optimal k is the one with highest silhouette score
optimal_k_kmeans = np.argmax(sil_scores) + 2
print("Optimal k for KMeans:", optimal_k_kmeans)
```

```
Optimal k for KMeans: 4
```

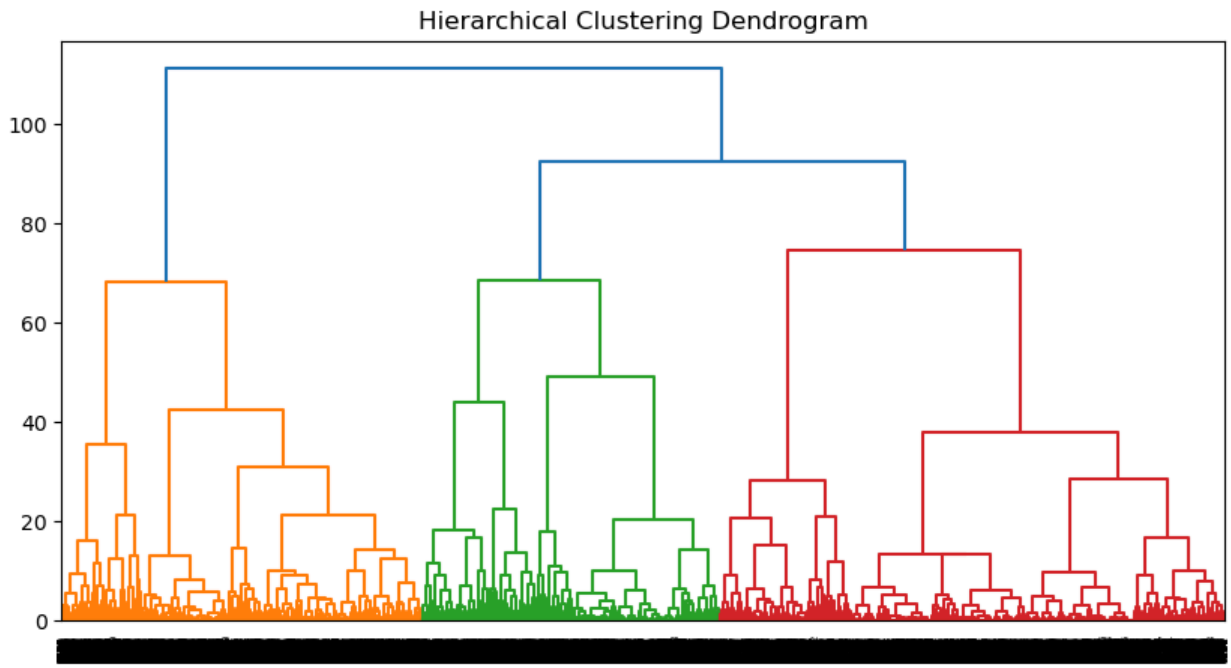
In [130…

```
# Fit final KMeans model
kmeans = KMeans(n_clusters=optimal_k_kmeans, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)
```

HIERARCHICAL CLUSTERING

In [131…

```
#  Dendrogram
# Visual representation of hierarchical cluster merging

linked = linkage(X_scaled, method='ward')

plt.figure(figsize=(10,5))
dendrogram(linked)
plt.title("Hierarchical Clustering Dendrogram")
plt.show()
```

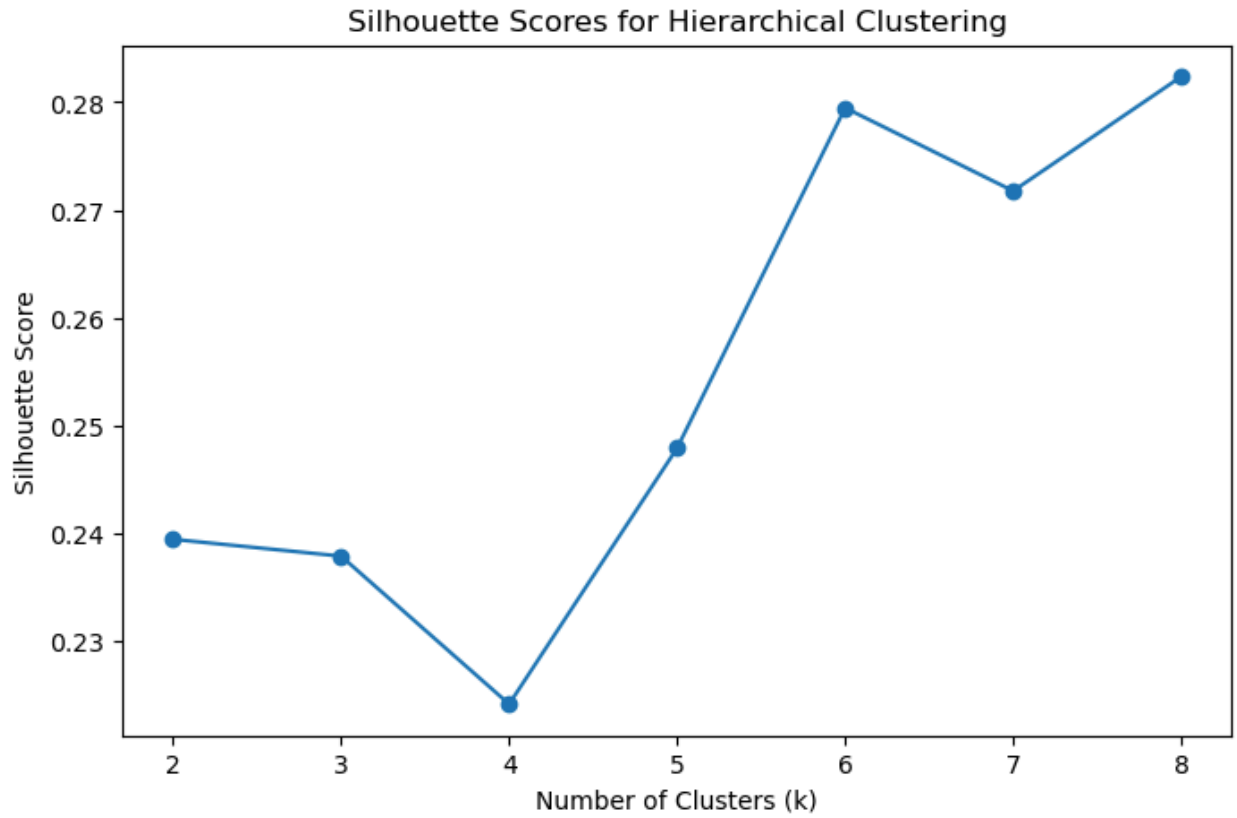


In [ ]:

In [133…

```
#  Silhouette Method for Hierarchical

hier_scores = []

for k in range(2, 9):
    agg = AgglomerativeClustering(n_clusters=k, linkage='ward')
    labels = agg.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    hier_scores.append(score)
    print(f"Hierarchical k={k}, Silhouette Score={score:.4f}")

plt.figure(figsize=(8,5))
plt.plot(range(2, 9), hier_scores, marker='o')
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Silhouette Score")
plt.title("Silhouette Scores for Hierarchical Clustering")
plt.show()
```

```
Hierarchical k=2, Silhouette Score=0.2395
Hierarchical k=3, Silhouette Score=0.2379
Hierarchical k=4, Silhouette Score=0.2242
Hierarchical k=5, Silhouette Score=0.2479
Hierarchical k=6, Silhouette Score=0.2795
Hierarchical k=7, Silhouette Score=0.2718
Hierarchical k=8, Silhouette Score=0.2824
```



In [ ]:

In [135…

```
optimal_k_hier = np.argmax(hier_scores) + 2
print("Optimal k for Hierarchical:", optimal_k_hier)
```

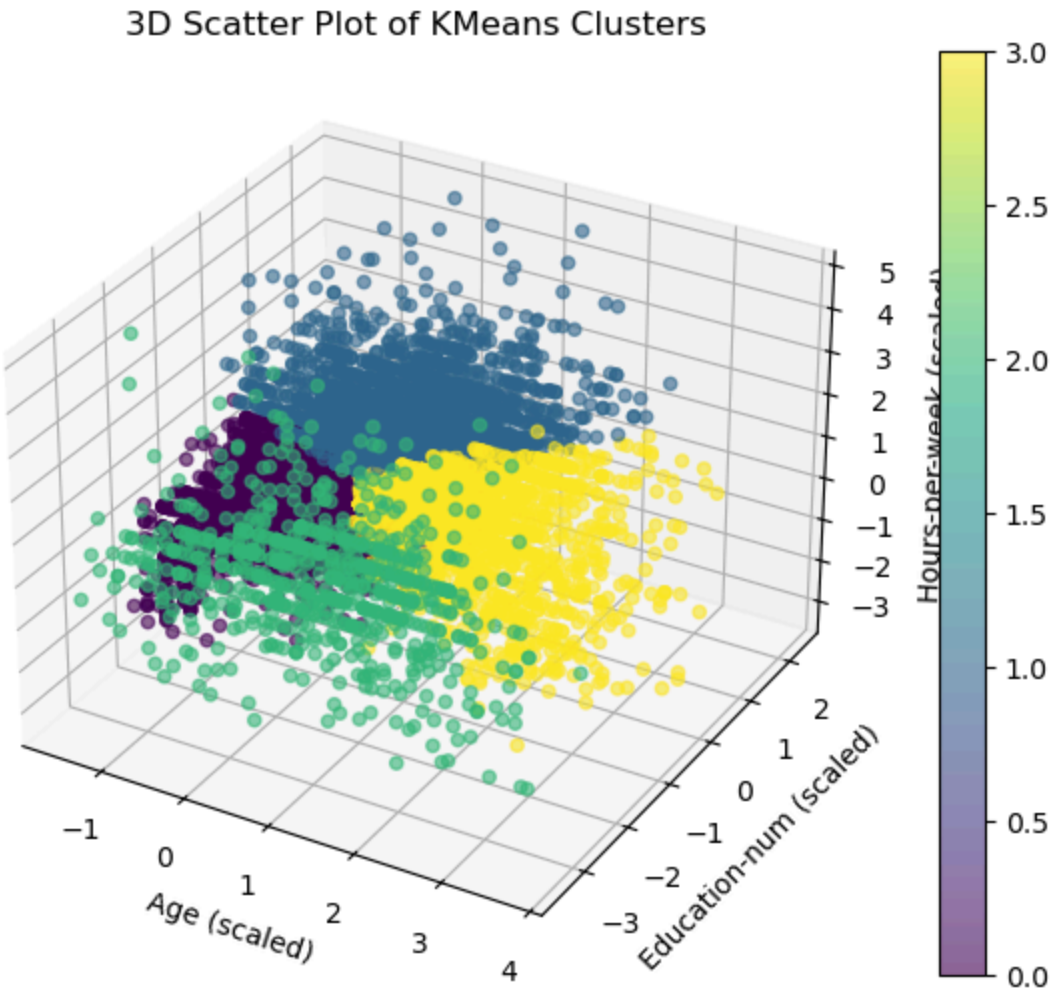
```
Optimal k for Hierarchical: 8
```

3D Scatter Plot for KMeans

In [149…

```
# Create 3D figure
fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

centroids = kmeans.cluster_centers_


# Plot clusters
scatter = ax.scatter(
    X_scaled[:, 0],   # age
    X_scaled[:, 1],   # education-num
    X_scaled[:, 2],   # hours-per-week
    c=kmeans_labels,
    cmap='viridis',
    alpha=0.6
)

# Axis labels
```

```
ax.set_xlabel("Age (scaled)")
ax.set_ylabel("Education-num (scaled)")
ax.set_zlabel("Hours-per-week (scaled)")
ax.set_title("3D Scatter Plot of KMeans Clusters")

plt.colorbar(scatter)
plt.show()
```



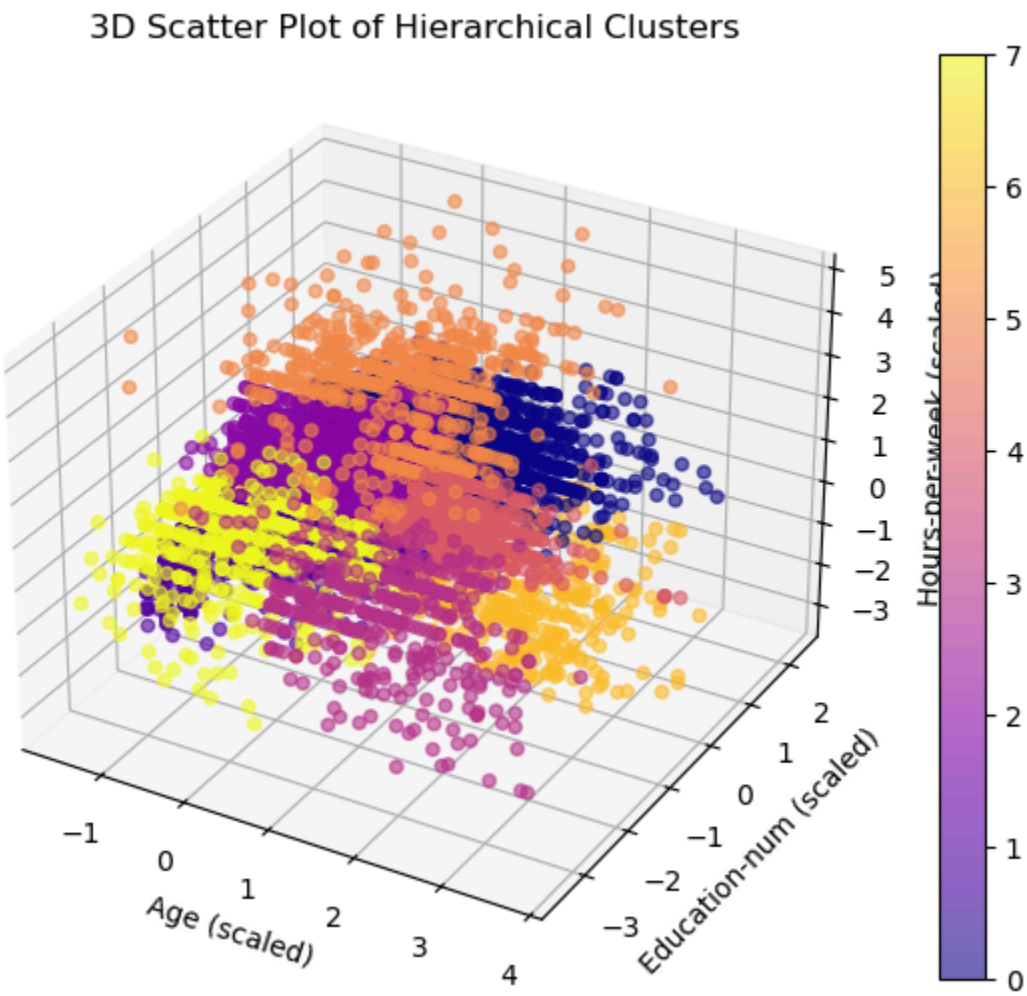
In [ ]:

3D Plot for Hierarchical Clustering

In [147…

```
# Fit hierarchical model using optimal k
agg = AgglomerativeClustering(n_clusters=optimal_k_hier)
hier_labels = agg.fit_predict(X_scaled)

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    X_scaled[:, 0],
```

```
    X_scaled[:, 1],
    X_scaled[:, 2],
    c=hier_labels,
    cmap='plasma',
    alpha=0.6
)

ax.set_xlabel("Age (scaled)")
ax.set_ylabel("Education-num (scaled)")
ax.set_zlabel("Hours-per-week (scaled)")
ax.set_title("3D Scatter Plot of Hierarchical Clusters")

plt.colorbar(scatter)
plt.show()
```



In [ ]:

############# WRITE YOUR REPORT IN THIS CELL (IF APPLICABLE)################### In this task, K-Means and Hierarchical clustering were applied to three numerical features: age, education-num, and hours-per-week. The data was standardised to ensure equal contribution of each feature to the distance calculations. The Elbow method and Silhouette score were used to determine the optimal number of clusters. For K-Means, the highest silhouette score was obtained at k = 4, indicating that four clusters provide the best separation and cohesion. For Hierarchical clustering, the highest silhouette score was observed at k = 8, although k = 4 produced a similar value. Overall, K-Means slightly outperformed Hierarchical clustering based on

silhouette score, suggesting clearer cluster separation in the selected feature space. To further examine cluster structure, 3D scatter plots were generated using the three selected features. The visualisations show moderate separation between clusters, although some overlap exists, which is consistent with the moderate silhouette scores. This suggests that while meaningful cluster structure is present, the boundaries between clusters are not sharply defined.

Task 4.2: Apply the PCA method to the dataset and extract the first two principal components (n_components=2). Plot the scatter plot of the dataset's first two components for the two classes of the income column (5%).

NOTE 1: You should comment on your code wherever necessary and briefly explain what the code is doing.

NOTE 2: You need to encode the categorical columns, normalise the dataset, and remove the income column before applying the PCA method.

In [118…

In [138…

```
# Replace '?' values with NaN
df.replace("?", np.nan, inplace=True)

# Drop rows with missing values
df2 = df.dropna()
```

In [ ]:

In [139…

```
#Encode Categorical Variables
encoder = OrdinalEncoder()
categorical_cols = df2.select_dtypes(include='object').columns

df2[categorical_cols] = encoder.fit_transform(df2[categorical_cols])
```

```
/var/folders/6b/x27jrjp92kn9r5_84c_ywfhcsgm8y7/T/ipykernel_55423/1630149546.p
y:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/sta
ble/user_guide/indexing.html#returning-a-view-versus-a-copy
  df2[categorical_cols] = encoder.fit_transform(df2[categorical_cols])
```

In [140…

```
# Separate features and target variable
```

```
X2 = df2.drop('income', axis=1)
y2 = df2['income']
```

In [ ]:

In [143…

```
#Normalise the Dataset

# PCA is affected by scale, so we normalise
scaler = MinMaxScaler()
X2_scaled = scaler.fit_transform(X2)
```

Apply PCA

In [144…

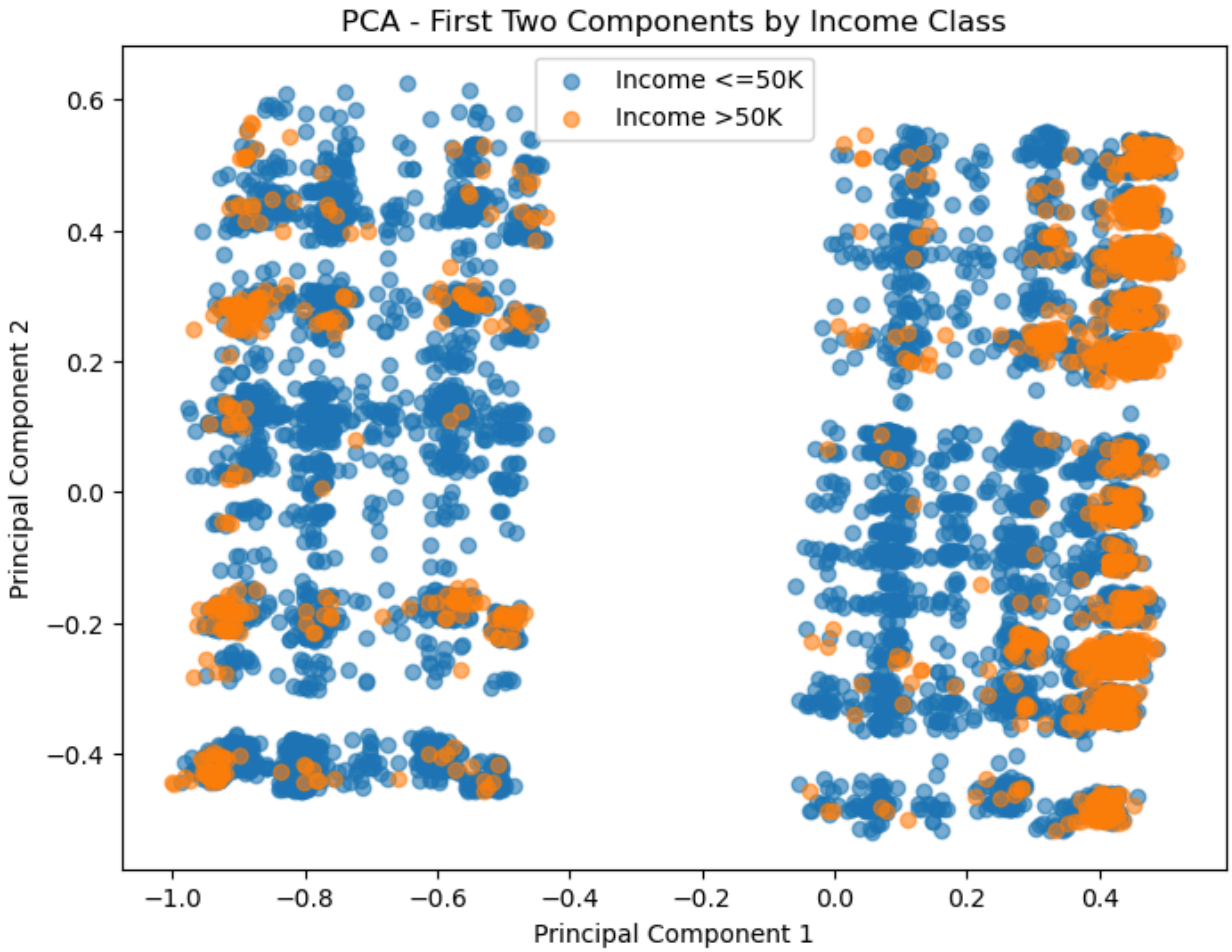
```
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X2_scaled)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

```
Explained Variance Ratio: [0.37235078 0.12949804]
```

In [ ]:

In [145…

```
#Plot PCA for the Two Income Classes
# Create masks for the two income classes
class_0 = y2 == 0
class_1 = y2 == 1

plt.figure(figsize=(8,6))

# Plot each class separately
plt.scatter(X_pca[class_0, 0], X_pca[class_0, 1], label="Income <=50K", alpha=

plt.scatter(X_pca[class_1, 0], X_pca[class_1, 1], label="Income >50K", alpha=0

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA - First Two Components by Income Class")
plt.legend()
plt.show()
```



############# WRITE YOUR REPORT IN THIS CELL (IF APPLICABLE)################### Before applying PCA, categorical variables were encoded and the dataset was normalised. The income column was removed prior to dimensionality reduction. The first two principal components explained approximately 50% of the total variance in the dataset. A scatter plot of these components, coloured by income class (≤50K and >50K), showed noticeable overlap between the two classes. This suggests that income is not clearly separable in two-dimensional space and may depend on more complex feature interactions.

In [ ]:

7CS033 Assignment Task 5

Complete the following tasks.

You should comment on your code where appropriate. Failing to add comments may result in a mark deduction.

You should write any explainations in the report cell if the task explicitly asks for this or if you wish to expand upon your solution.

Ensure you have ran your code cell and printed any of the required outputs. If necessary you can use more than one code cell.

T5.1: Download the 'Portugal_online_retail', 'Sweden_online_retail, and 'UK_online_retail' datasets from Canvas. Apply the apriori algorithm to all datasets using three different confidence levels. Select one confidence level for each dataset that you think works better. Determine the first three most important rules for each dataset using the selected confidence level and report them in the report cell. Explain what each rule means (Completing the report cell is required) (10%).

NOTE: You should comment on your code

In [23]:
```
############# WRITE YOUR CODE IN THIS CELL (IF APPLICABLE)####################
import pandas as pd
import numpy as np
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
```

In [24]:
```
#import portugal, sweden and uk dataset
df_portugal = pd.read_csv('Portugal_online_retail.csv', dtype={'InvoiceNo': st
df_sweden = pd.read_csv('Sweden_online_retail.csv', dtype={'InvoiceNo': str})
df_uk = pd.read_csv('UK_online_retail.csv', dtype={'InvoiceNo': str})
```

In [25]:
```
#Drop the InvoiceNo column
df_portugal = df_portugal.drop('InvoiceNo', axis =1);
df_sweden = df_sweden.drop('InvoiceNo', axis =1);
df_uk = df_uk.drop('InvoiceNo', axis =1);
```

In [26]:
```
# Convert 0/1 to boolean (VERY IMPORTANT)
df_portugal = df_portugal.astype(bool)
df_sweden = df_sweden.astype(bool)
df_uk = df_uk.astype(bool)
```

In [27]:
```
print(f"{df_portugal.shape[0]} = Portugal rows!")
print(f"{df_sweden.shape[0]} = Sweden rows!")
print(f"{df_uk.shape[0]} = UK rows!")
```

```
58 = Portugal rows!
36 = Sweden rows!
18667 = UK rows!
```

In [6]:
```
#Since Portugal and Sweden are very small datasets, we must use lower support/
#For UK (large dataset), we can use higher thresholds.
```

PORTUGAL - RUN APRIORI (Different support per dataset)

In [7]:
```
#For Small datase, support cannot be too high (you will lose patterns)
#For Large dataset, support must be lower to capture frequent combinations
```

GENERATE FREQUENCY WITH 3 DIFFERENT SUPPORT LEVELS

In [8]:
```
# Test three support values
frequent_p_01 = apriori(df_portugal, min_support=0.04, use_colnames=True);
frequent_p_05 = apriori(df_portugal, min_support=0.05, use_colnames=True);
frequent_p_10 = apriori(df_portugal, min_support=0.10, use_colnames=True);
```

In [9]:
```
print("Number of Frequent Itemsets for 0.04:", len(frequent_p_01))
print("Number of Frequent Itemsets for 0.05:", len(frequent_p_05))
print("Number of Frequent Itemsets for 0.10:", len(frequent_p_10))
```

```
Number of Frequent Itemsets for 0.04: 7446
Number of Frequent Itemsets for 0.05: 7446
Number of Frequent Itemsets for 0.10: 110
```

In [10]:
```
#For the Portugal dataset, a support of 0.10 is best because it produces a man
#meaningful itemsets, while lower support values generate too many noisy items
```

In [ ]:

GENERATE RULES WITH 3 DIFFERENT CONFIDENCE LEVELS

In [11]:
```
# Generate association rules (try different confidence levels)
rules_03 = association_rules(frequent_p_10, metric="confidence", min_threshold
rules_05 = association_rules(frequent_p_10, metric="confidence", min_threshold
rules_07 = association_rules(frequent_p_10, metric="confidence", min_threshold

print("Confidence 0.3:", len(rules_03))
print("Confidence 0.5:", len(rules_05))
print("Confidence 0.7:", len(rules_07))
```

```
Confidence 0.3: 317
Confidence 0.5: 300
Confidence 0.7: 209
```

In [12]:
```
#The best association rules for Portugal are those with a confidence threshold
#reliability and quantity, capturing meaningful patterns without being too noi
```

In [13]:
```
# Select confidence = 0.5
rules_portugal = rules_05

# Sort by lift (higher lift = stronger rule)
top_portugal = rules_portugal.sort_values(by='lift', ascending=False).head(3)
top_portugal
```

Out[13]:

	antecedents	consequents	antecedent support	consequent support	suppor
58	frozenset({SCANDINAVIAN PAISLEY PICNIC BAG})	frozenset({PINK VINTAGE PAISLEY PICNIC BAG})	0.120690	0.137931	0.12069
263	frozenset({JUMBO BAG SCANDINAVIAN BLUE PAISLEY...	frozenset({LUNCH BAG PINK POLKADOT, JUMBO SHOP...	0.137931	0.103448	0.10344
264	frozenset({JUMBO BAG SCANDINAVIAN BLUE PAISLEY...	frozenset({JUMBO BAG PINK VINTAGE PAISLEY, LUN...	0.137931	0.103448	0.10344

SWEDEN - RUN APRIORI (Different support per dataset)

GENERATE FREQUENCY WITH 3 DIFFERENT SUPPORT LEVELS

In [14]:
```
# Test three support values

frequent_s_08 = apriori(df_sweden, min_support=0.08, use_colnames=True)
frequent_s_10 = apriori(df_sweden, min_support=0.10, use_colnames=True)
frequent_s_15 = apriori(df_sweden, min_support=0.15, use_colnames=True)

print("Support 0.08:", len(frequent_s_08))
print("Support 0.10:", len(frequent_s_10))
print("Support 0.15:", len(frequent_s_15))
```

```
Support 0.08: 118
Support 0.10: 25
Support 0.15: 3
```

In [15]:
```
#0.10 provided a balanced and meaningful number of itemsets (25) while 0.08 pr
#frequent itemsets (118), indicating noise, while 0.15 produced too few (3)
```

GENERATE RULES WITH 3 DIFFERENT CONFIDENCE LEVELS

In [16]:
```
# Generate association rules (try different confidence levels)
rules_s_04 = association_rules(frequent_s_10, metric="confidence", min_thresho
rules_s_05 = association_rules(frequent_s_10,  metric="confidence", min_thresh
rules_s_07 = association_rules(frequent_s_10, metric="confidence", min_thresho


print("Confidence 0.4:", len(rules_s_04))
print("Confidence 0.5:", len(rules_s_05))
print("Confidence 0.7:", len(rules_s_07))
```

```
Confidence 0.4: 6
Confidence 0.5: 6
Confidence 0.7: 6
```

In [17]:
```
#0.7 Confidence was selected because all six rules remained the same across 0.
#rules are already strong, so using the higher threshold ensures greater relia
```

In [ ]:

In [18]:
```
# Select confidence = 0.5
rules_sweden = rules_s_05

# Sort by lift (higher lift = stronger rule)
top_sweden= rules_sweden.sort_values(by='lift', ascending=False).head(3)
top_sweden
```

Out[18]:

	antecedents	consequents	antecedent support	consequent support	support	c
1	frozenset({PACK OF 72 RETROSPOT CAKE CASES})	frozenset({PACK OF 60 SPACEBOY CAKE CASES})	0.111111	0.111111	0.111111	
2	frozenset({PACK OF 60 SPACEBOY CAKE CASES})	frozenset({PACK OF 72 RETROSPOT CAKE CASES})	0.111111	0.111111	0.111111	
0	frozenset({GUMBALL COAT RACK})	frozenset({POSTAGE})	0.138889	0.611111	0.138889	

In [ ]:

UK - RUN APRIORI (Different support per dataset)

GENERATE FREQUENCY WITH 3 DIFFERENT SUPPORT LEVELS

In [22]:
```
### UK - TEST SUPPORT VALUES
frequent_uk_009 = apriori(df_uk, min_support=0.009, use_colnames=True)
frequent_uk_01 = apriori(df_uk, min_support=0.01, use_colnames=True)
frequent_uk_02 = apriori(df_uk, min_support=0.02, use_colnames=True)
```

```
print("Support 0.009:", len(frequent_uk_009))
print("Support 0.01:", len(frequent_uk_01))
print("Support 0.02:", len(frequent_uk_02))
```

```
Support 0.009: 2606
Support 0.01: 1970
Support 0.02: 369
```

In [ ]:
```
#Support = 0.01 is the best choice because it captures meaningful and recurrin
#too restrictive or generating excessive noise. Lower support includes too man
#while higher support removes useful associations
```

In [ ]:

GENERATE RULES WITH 3 DIFFERENT CONFIDENCE LEVELS

In [28]:
```
rules_uk_04 = association_rules(frequent_uk_01, metric="confidence", min_thres
rules_uk_05 = association_rules(frequent_uk_01, metric="confidence", min_thres
rules_uk_06 = association_rules(frequent_uk_01, metric="confidence", min_thres

print("Confidence 0.4:", len(rules_uk_04))
print("Confidence 0.5:", len(rules_uk_05))
print("Confidence 0.6:", len(rules_uk_06))
```

```
Confidence 0.4: 1502
Confidence 0.5: 1055
Confidence 0.6: 667
```

In [29]:
```
# Select confidence = 0.5
rules_uk = rules_uk_05

# Sort by lift (higher lift = stronger rule)
top_uk= rules_uk.sort_values(by='lift', ascending=False).head(3)
top_uk
```

Out[29]:

	antecedents	consequents	antecedent support	consequent support	support	confide
501	frozenset({HERB MARKER PARSLEY, HERB MARKER RO...	frozenset({HERB MARKER THYME})	0.011089	0.012321	0.010553	0.951
506	frozenset({HERB MARKER THYME})	frozenset({HERB MARKER PARSLEY, HERB MARKER RO...	0.012321	0.011089	0.010553	0.856
496	frozenset({HERB MARKER MINT, HERB MARKER THYME})	frozenset({HERB MARKER ROSEMARY})	0.010714	0.012375	0.010232	0.955

In [ ]:

######################### Write your report in this cell (if applicable) ##############################

PORTUGAL DATASET For the Portugal dataset, a support of 0.10 is best because it produces a manageable number of meaningful itemsets, while lower support values generate too many noisy itemsets

The best association rules for Portugal are those with a confidence threshold of 0.5 because they balance reliability and quantity, capturing meaningful patterns without being too noisy or too restrictive.

SWEDEN DATASET 0.10 was chosen as it provided a balanced and meaningful number of itemsets (25) while 0.08 produced too many

frequent itemsets (118), indicating noise, while 0.15 produced too few (3)

0.5 is a better confidence threshold because 50% confidence means the rule is correct at least half the time, which gives good reliability without removing too many rules

UK DATASET Support = 0.01 is the best choice because it captures meaningful and recurring purchasing patterns without being too restrictive or generating excessive noise. Lower support includes too many insignificant combinations, while higher support removes useful associations

Confidence = 0.5 is appropriate because it ensures the rules are reliable and strong enough to be meaningful, while still retaining a sufficient number of patterns for analysis

In [ ]:

In [ ]:

7CS033 Assignment Task 6

Complete the following tasks.

You should comment on your code where appropriate. Failing to add comments may result in a mark deduction.

You should write any explainations in the report cell if the task explicitly asks for this or if you wish to expand upon your solution.

Ensure you have ran your code cell and printed any of the required outputs. If necessary you can use more than one code cell.

Task 6.1: Download the 'Tweets' dataset from Canvas. Classify the sentiments in the dataset using six classifiers and calculate all evaluation metrics (10%).

NOTE: If the running time is too long, you can reduce the number of samples.

NOTE: You should comment on your code and explain what each part is doing

NOTE: You should improve the model's performance as much as possible

In [29]:
```
############# WRITE THE CODE IN THIS CELL ####################
import pandas as pd
import numpy as np


# Text preprocessing
import re
import string

# Sklearn tools
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer

from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
```

```
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)

from sklearn import metrics #metrics


# Six classifiers
from sklearn.linear_model import LogisticRegression
#from sklearn.naive_bayes import MultinomialNB
#from sklearn.svm import LinearSVC
#from sklearn.tree import DecisionTreeClassifier
#from sklearn.ensemble import RandomForestClassifier
#from sklearn.neighbors import KNeighborsClassifier


from sklearn import svm
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB


# Remove Stopwords
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

import matplotlib.pyplot as plt
```

In [2]:

```
df = pd.read_csv('Tweets.csv')
df.head(10)
```

Out[2]:

	sentiment	tweet
0	0	Eating a gyro for breakfast at the airport. Pr...
1	1	Well .. .you fall of the 'horse' and get strai...
2	0	thanks for the responses but when is it really...
3	0	depressed.! fuuck my life!
4	1	is LMAO, just thinkin of the 80/20 rule.....wo...
5	1	Going out to eat neenahhhbabe(:
6	0	is saying goodbye to her momma for 4 months..
7	0	@AprilMaiya I'm not!
8	0	at work missin my brandi
9	1	@abygurl yeah i know i can't wait too

In [3]:

```
print(df.shape)
print(df.isna().sum())
```

```
(40000, 2)
sentiment    0
tweet        0
dtype: int64
```

In [4]:

```
# Keep only required columns
df = df[['tweet', 'sentiment']]
df
```

Out[4]:

	tweet	sentiment
0	Eating a gyro for breakfast at the airport. Pr...	0
1	Well .. .you fall of the 'horse' and get strai...	1
2	thanks for the responses but when is it really...	0
3	depressed.! fuuck my life!	0
4	is LMAO, just thinkin of the 80/20 rule.....wo...	1
...	...	...
39995	@TafiApple Awww... you retweeted me! Thanks!	1
39996	Getting some coffee before I faceplant. Guess ...	1
39997	@bernderk it could *not* be worse!	0
39998	I thought about getting a bunny...but my beagl...	1
39999	So I have my new phone. Now I just have no con...	0

40000 rows × 2 columns

In [5]:
```
def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+", "", text)  # remove URLs
    text = re.sub(r"@\w+", "", text)     # remove mentions
    text = re.sub(r"#\w+", "", text)     # remove hashtags
    text = text.translate(str.maketrans('', '', string.punctuation))
    return text

# Apply cleaning
df['clean_tweet'] = df['tweet'].apply(clean_text)
df
```

Out[5]:

	tweet	sentiment	clean_tweet
0	Eating a gyro for breakfast at the airport. Pr...	0	eating a gyro for breakfast at the airport pro...
1	Well .. .you fall of the 'horse' and get strai...	1	well you fall of the horse and get straight b...
2	thanks for the responses but when is it really...	0	thanks for the responses but when is it really...
3	depressed.! fuuck my life!	0	depressed fuuck my life
4	is LMAO, just thinkin of the 80/20 rule.....wo...	1	is lmao just thinkin of the 8020 rulewonder ho...
...	...	...	...
39995	@TafiApple Awww... you retweeted me! Thanks!	1	awww you retweeted me thanks
39996	Getting some coffee before I faceplant. Guess ...	1	getting some coffee before i faceplant guess i...
39997	@bernderk it could *not* be worse!	0	it could not be worse
39998	I thought about getting a bunny...but my beagl...	1	i thought about getting a bunnybut my beagle g...
39999	So I have my new phone. Now I just have no con...	0	so i have my new phone now i just have no cont...

40000 rows × 3 columns

In [ ]:

In [6]:
```
# Download resources (run once)
nltk.download('punkt')
nltk.download('stopwords')

# Load English stopwords
stop_words = set(stopwords.words('english'))
print(stop_words)
```

```
{'down', 'once', 'ours', 'who', 'didn', 'more', 'few', "he'd", 'him', 'can',
'd', 'his', 'how', "you'd", 'itself', 'or', 'while', 'having', 'after', "must
n't", 'those', 'theirs', 't', 'such', 'is', "i've", 'y', 'isn', "i'd", 'over',
'this', "she'll", 'shan', "isn't", 'ourselves', 'as', 'in', 'am', 'they', 'he',
'some', 'them', "they'll", 'wouldn', "wouldn't", 'should', "didn't", 'above',
'at', 'had', 'before', 'shouldn', 'the', 'your', 'doing', "we're", 'don', 'unde
r', 'weren', 'hers', "you've", 'both', 'to', 'needn', "couldn't", "hasn't", "sh
ouldn't", 'each', 'll', 'their', 'herself', "we'd", 'own', "needn't", 'yoursel
f', 'between', 'hadn', "i'll", 'if', 'was', 'further', 'be', 'just', 'then', 'm
e', 'through', 'you', 'she', 'that', 'against', 'no', 're', 'hasn', 'wasn', "th
ey've", 'when', "doesn't", 's', 'yours', 've', 'nor', "it'll", 'aren', 'any',
'only', 'off', 'same', "he'll", "we've", 'it', "he's", 'on', 'so', 'out', "yo
u're", 'which', 'has', 'again', 'm', 'not', "aren't", 'i', 'here', 'from', 'mig
htn', "should've", 'all', "you'll", 'myself', 'ma', "we'll", 'her', "that'll",
'yourselves', 'o', 'than', 'of', 'will', 'now', 'into', 'there', 'and', "do
n't", 'mustn', 'up', "she's", 'won', 'because', 'couldn', "wasn't", 'our', 'to
o', 'we', 'being', 'these', "they're", 'themselves', 'whom', 'been', "won't",
'until', 'by', "they'd", 'have', 'below', 'himself', "she'd", "weren't", 'wher
e', 'but', 'most', 'did', 'other', 'a', "mightn't", 'an', 'does', 'very', 'wit
h', "hadn't", 'what', 'about', 'haven', "haven't", "it's", 'doesn', 'my', 'wh
y', 'ain', "i'm", 'its', 'during', "it'd", "shan't", 'do', 'are', 'for', 'wer
e'}
```

```
[nltk_data] Downloading package punkt to /Users/oeze/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /Users/oeze/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

In [ ]:

In [7]:

```python
#function to Tokenize and remove stopwords
def tokenize_stopwords_text(text):
    # Tokenize
    tokens = word_tokenize(text)

    # Remove stopwords
    filtered_tokens = [word for word in tokens if word not in stop_words]

    # Join back to string
    return " ".join(filtered_tokens)
```

In [8]:

```python
#df['tokenize_stopwords'] = df['clean_tweet'].apply(clean_text)
df['tokenize_stopwords'] = df['clean_tweet'].apply(lambda x:" ".join(i for i i
df
```

Out[8]:

	tweet	sentiment	clean_tweet	tokenize_stopwords
0	Eating a gyro for breakfast at the airport. Pr...	0	eating a gyro for breakfast at the airport pro...	eating gyro breakfast airport probly gonna lun...
1	Well .. .you fall of the 'horse' and get strai...	1	well you fall of the horse and get straight b...	well fall horse get straight back probably dri...
2	thanks for the responses but when is it really...	0	thanks for the responses but when is it really...	thanks responses really lol say today say tomo...
3	depressed.! fuuck my life!	0	depressed fuuck my life	depressed fuuck life
4	is LMAO, just thinkin of the 80/20 rule.....wo...	1	is lmao just thinkin of the 8020 rulewonder ho...	lmao thinkin 8020 rulewonder shes gonna like 2...
...	...	...	...	...
39995	@TafiApple Awww... you retweeted me! Thanks!	1	awww you retweeted me thanks	awww retweeted thanks
39996	Getting some coffee before I faceplant. Guess ...	1	getting some coffee before i faceplant guess i...	getting coffee faceplant guess wont sleeping t...
39997	@bernderk it could *not* be worse!	0	it could not be worse	could worse
39998	I thought about getting a bunny...but my beagl...	1	i thought about getting a bunnybut my beagle g...	thought getting bunnybut beagle goes berzerk c...
39999	So I have my new phone. Now I just have no con...	0	so i have my new phone now i just have no cont...	new phone contacts hopefully get done tonight

40000 rows × 4 columns

In [ ]:

In [9]:

```
# create an input variable 'X' of the 'tokenize_stopwords' feature and target
# Using the sklearn CountVectorizer() method to convert text data in X into nu

X = df['tokenize_stopwords']
y = df['sentiment']

#we use countvectorizer to convert the text into a sparse matrix of counts.
#This will enable us to train ML models on our text data as it will be represe

vectorizer = CountVectorizer(token_pattern = r'\b\w+\b')
#the token pattern tells wordvectorizer how the text should be tokenized.
```

```
#here we specified the tokens will start at a word boundry '\b' then must cont
#note that set up this way words of length 1 will be ignored

X = vectorizer.fit_transform(X)

#below I convert this into a dataframe just so we can see what the count vecto
vectorized_df = pd.DataFrame(X.toarray(), columns = vectorizer.get_feature_nam
print(vectorized_df.columns)
vectorized_df
```

```
Index(['0', '00', '0006', '001', '0010utc', '0012', '0030', '01',
       '011203present', '02',
       ...
       'øª', 'øªø', 'øªù', 'ø³ø', 'ø³ù', 'ø¹', 'ø¹ø', 'ø¹ù', 'ù', 'ùø'],
      dtype='str', length=36814)
```

Out[9]:

	0	00	0006	001	0010utc	0012	0030	01	011203present	02	...	øª
0	0	0	0	0	0	0	0	0	0	0	...	0
1	0	0	0	0	0	0	0	0	0	0	...	0
2	0	0	0	0	0	0	0	0	0	0	...	0
3	0	0	0	0	0	0	0	0	0	0	...	0
4	0	0	0	0	0	0	0	0	0	0	...	0
...	...	...	...	...	...	...	...	...	...	...	...	...
39995	0	0	0	0	0	0	0	0	0	0	...	0
39996	0	0	0	0	0	0	0	0	0	0	...	0
39997	0	0	0	0	0	0	0	0	0	0	...	0
39998	0	0	0	0	0	0	0	0	0	0	...	0
39999	0	0	0	0	0	0	0	0	0	0	...	0

40000 rows × 36814 columns

In [ ]:

In [10]:
```
#train six classifiers to predict the sentiment of a review summary.
```

In [19]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3, ran


SVM = svm.SVC()
RF = RandomForestClassifier()
KNN = KNeighborsClassifier()
DT = DecisionTreeClassifier()
NB = GaussianNB()
LR = LogisticRegression()

#train the models
```

```
SVM.fit(X_train, y_train)
RF.fit(X_train, y_train)
KNN.fit(X_train, y_train)
DT.fit(X_train, y_train)
LR.fit(X_train,y_train)
NB.fit(X_train.toarray(),y_train)
```

Out[19]:

▾ GaussianNB

Parameters

priors	None
var_smoothing	1e-09

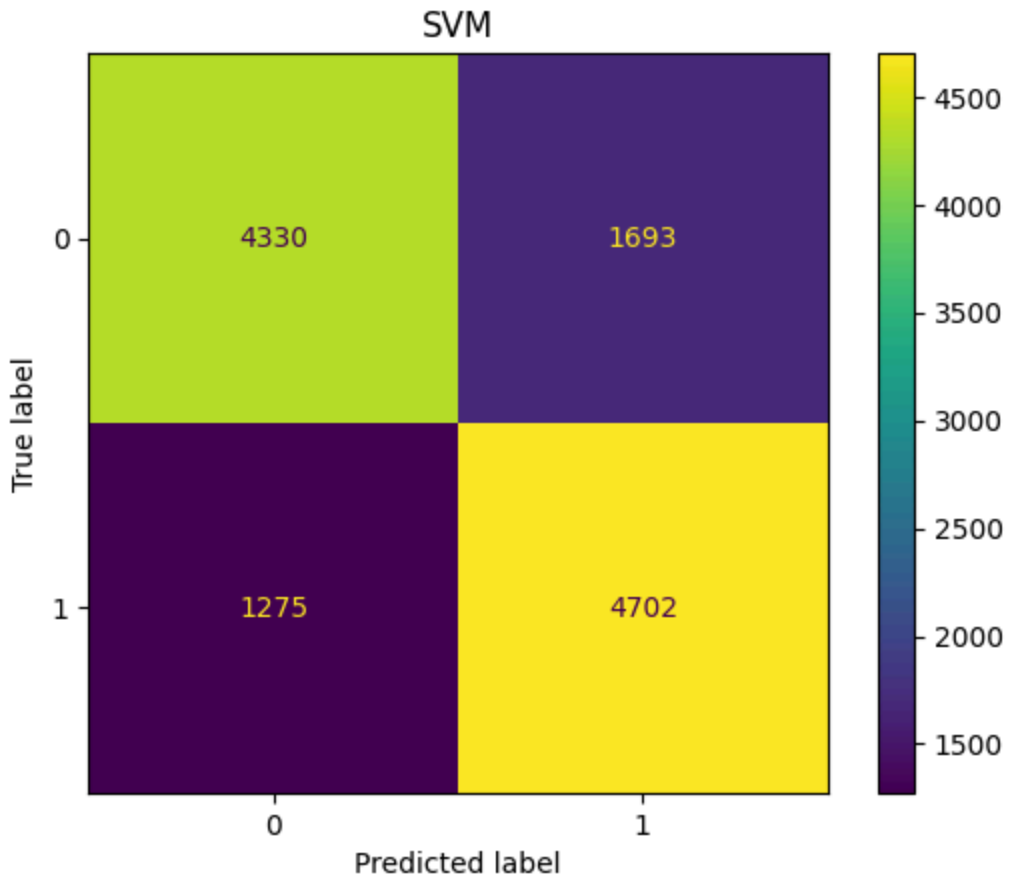
In [ ]:

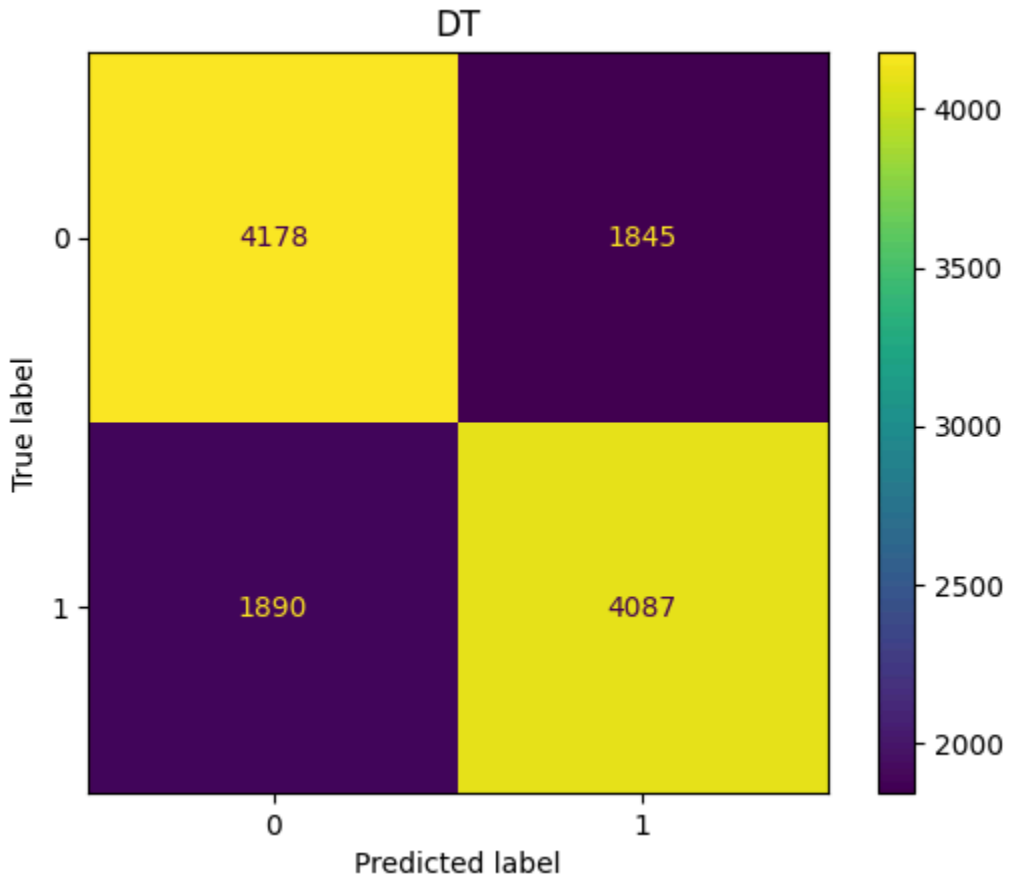
In [20]:

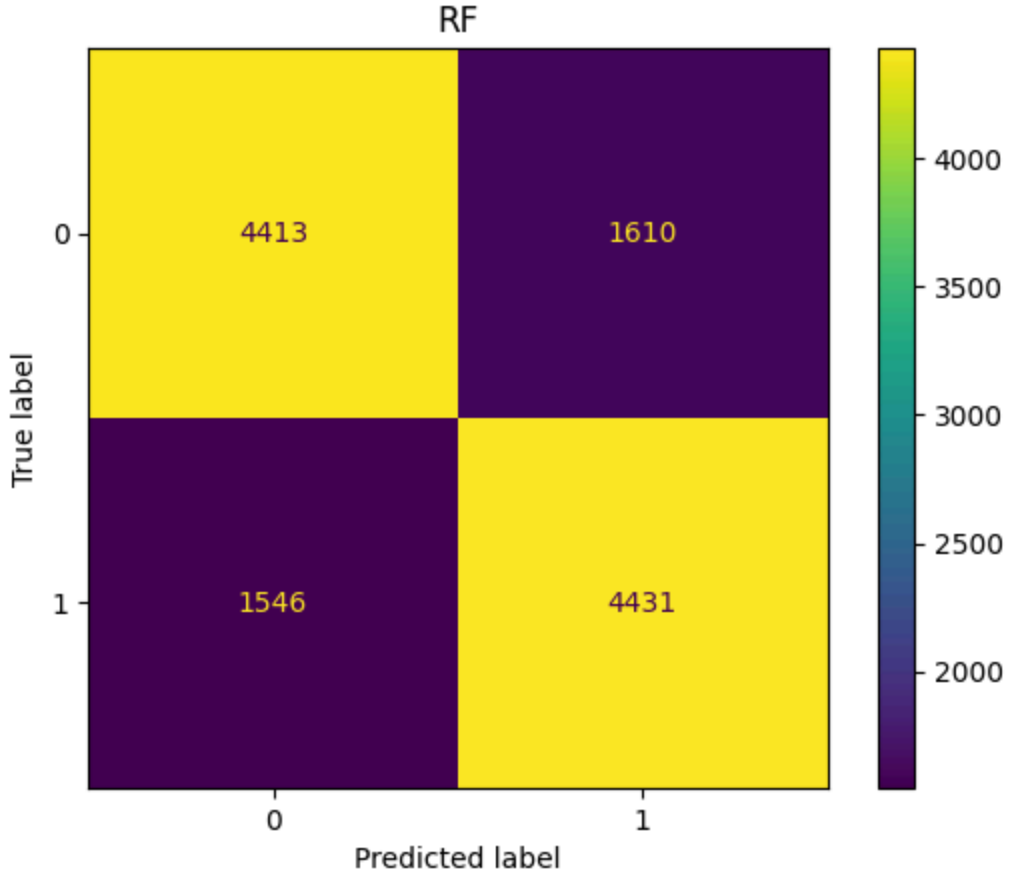
```
#first make predictions using our trained models on the test data

SVM_pred = SVM.predict(X_test)
DT_pred = DT.predict(X_test)
RF_pred = RF.predict(X_test)
KNN_pred = KNN.predict(X_test)

#Now we can generate the confusion matrices



SVM_cm = confusion_matrix(y_test, SVM_pred, labels = SVM.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix = SVM_cm, display_labels = SVM.
disp.plot()
plt.title('SVM')

DT_cm = confusion_matrix(y_test, DT_pred, labels = DT.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix = DT_cm, display_labels = DT.cl
disp.plot()
plt.title('DT')

RF_cm = confusion_matrix(y_test, RF_pred, labels = RF.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix = RF_cm, display_labels = RF.cl
disp.plot()
plt.title('RF')

KNN_cm = confusion_matrix(y_test, KNN_pred, labels = KNN.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix = KNN_cm, display_labels = KNN.
disp.plot()
plt.title('KNN')
```

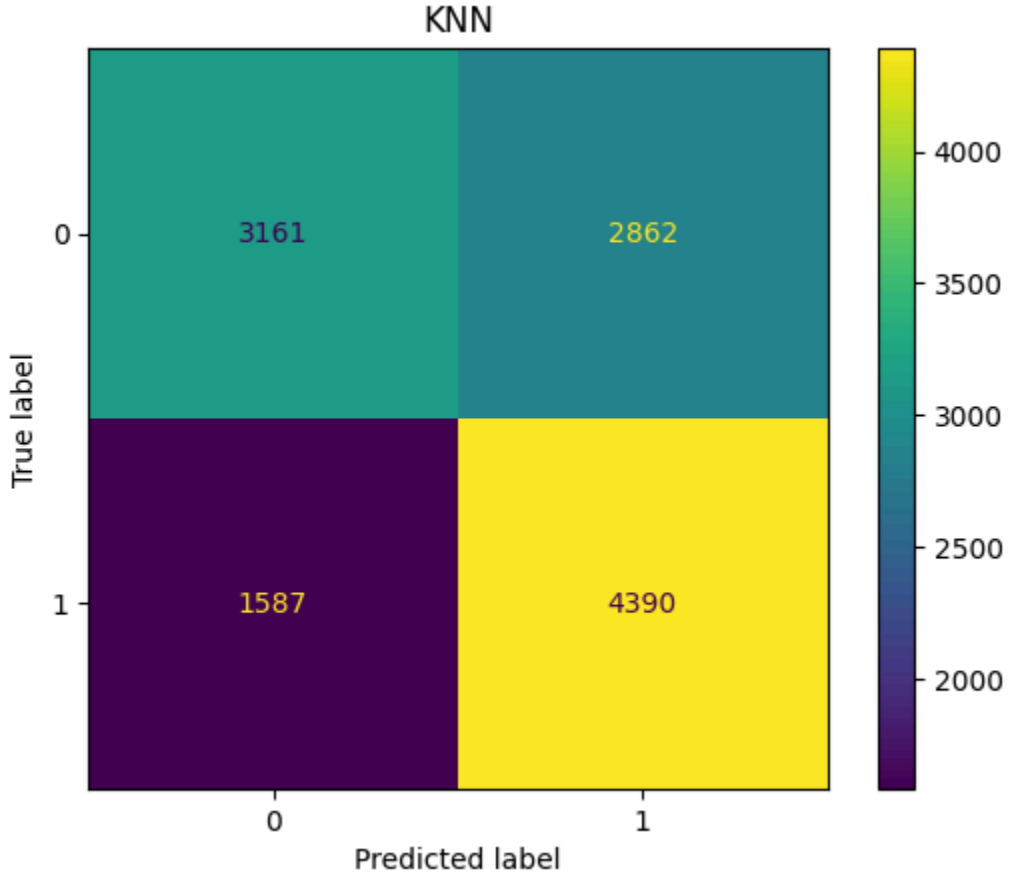
Out[20]:

```
Text(0.5, 1.0, 'KNN')
```


SVM
4330
1693
1275
4702
True label
Predicted label
0
1
4500
4000
3500
3000
2500
2000
1500


DT
4178
1845
1890
4087
True label
Predicted label
0
1
4000
3500
3000
2500
2000


RF
4413
1610
1546
4431
0
1
True label
0
1
Predicted label
4000
3500
3000
2500
2000


KNN
3161
2862
1587
4390
0
1
True label
0
1
Predicted label
4000
3500
3000
2500
2000

In [ ]:

In [21]:
```
#Next we will write a function that takes in the confusion matrices we generat
```

In [33]:
```
#function to return the
def getSpecifity(y,prediction):

    cm = metrics.confusion_matrix(y, prediction)

    #Extract Values
    TN = cm[0][0]
    FP = cm[0][1]
    FN = cm[1][0]
    TP = cm[1][1]

    # Specificity
    specificity = TN / (TN + FP)
    return specificity
```

In [36]:
```
print('SVM Metrics \n')

#Get Confusion Matrix

print("Specificity: %.2f" % getSpecifity(y_test, SVM_pred))
misclassification = 1 - metrics.accuracy_score(y_test, SVM_pred)
print('Misclassification Rate: %.2f' % misclassification)
print('Accuracy: %.2f' %metrics.accuracy_score(y_test, SVM_pred))
print('Precision: %.2f' %metrics.precision_score(y_test, SVM_pred))
print('Recall: %.2f' %metrics.recall_score(y_test, SVM_pred))
print('F1 Score: %.2f \n' %metrics.f1_score(y_test, SVM_pred))

print('DT Metrics \n')
print("Specificity: %.2f" % getSpecifity(y_test, DT_pred))
misclassification = 1 - metrics.accuracy_score(y_test, DT_pred)
print('Misclassification Rate: %.2f' % misclassification)
print('Accuracy: %.2f' %metrics.accuracy_score(y_test, DT_pred))
print('Precision: %.2f' %metrics.precision_score(y_test, DT_pred))
print('Recall: %.2f' %metrics.recall_score(y_test, DT_pred))
print('F1 Score: %.2f \n' %metrics.f1_score(y_test, DT_pred))

print('RF Metrics \n')
print("Specificity: %.2f" % getSpecifity(y_test, RF_pred))
misclassification = 1 - metrics.accuracy_score(y_test, RF_pred)
print('Misclassification Rate: %.2f' % misclassification)
print('Accuracy: %.2f' %metrics.accuracy_score(y_test, RF_pred))
print('Precision: %.2f' %metrics.precision_score(y_test, RF_pred))
print('Recall: %.2f' %metrics.recall_score(y_test, RF_pred))
print('F1 Score: %.2f \n' %metrics.f1_score(y_test, RF_pred))

print('KNN Metrics \n')
print("Specificity: %.2f" % getSpecifity(y_test, KNN_pred))
misclassification = 1 - metrics.accuracy_score(y_test, KNN_pred)
print('Misclassification Rate: %.2f' % misclassification)
```

```
print('Accuracy: %.2f' %metrics.accuracy_score(y_test, KNN_pred))
print('Precision: %.2f' %metrics.precision_score(y_test, KNN_pred))
print('Recall: %.2f' %metrics.recall_score(y_test, KNN_pred))
print('F1 Score: %.2f \n' %metrics.f1_score(y_test, KNN_pred))
```

```
SVM Metrics

Specificity: 0.72
Misclassification Rate: 0.25
Accuracy: 0.75
Precision: 0.74
Recall: 0.79
F1 Score: 0.76

DT Metrics

Specificity: 0.69
Misclassification Rate: 0.31
Accuracy: 0.69
Precision: 0.69
Recall: 0.68
F1 Score: 0.69

RF Metrics

Specificity: 0.73
Misclassification Rate: 0.26
Accuracy: 0.74
Precision: 0.73
Recall: 0.74
F1 Score: 0.74

KNN Metrics

Specificity: 0.52
Misclassification Rate: 0.37
Accuracy: 0.63
Precision: 0.61
Recall: 0.73
F1 Score: 0.66
```

############ WRITE YOUR REPORT IN THIS CELL (IF APPLICABLE) #############

The Tweets dataset contains labelled tweets for sentiment analysis. We cleaned the text by converting it to lowercase and removing URLs, mentions, hashtags, and punctuation. Stopwords were then removed from the cleaned tweets.

The processed text was converted into numerical features using CountVectorizer. Six classifiers were trained, and metrics such as Accuracy, Precision, Recall, F1-score, Misclassification Rate, and Specificity were used to evaluate and compare

their performance.

SVM performed the best overall. It has the highest accuracy (0.75), F1 score (0.76), and a good balance between recall (0.79) and specificity (0.72), meaning it detects positives well without misclassifying too many negatives.

Random Forest is very close to SVM but slightly lower on most metrics.

Decision Tree performs moderately but not as strong as SVM or RF.

KNN performs the worst. Although its recall is reasonable (0.73), its specificity is very low (0.52), meaning it wrongly classifies many negative cases as positive.

Overall, SVM is the most reliable model among the four.

In [ ]: